Lucas de Oliveira Hernandez

Otimização de Modelos de Aprendizado de Máquina para Detecção de Intrusões em Redes

São Paulo - Brasil

2025

Lucas de Oliveira Hernandez

Otimização de Modelos de Aprendizado de Máquina para Detecção de Intrusões em Redes

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Prof. Me. Sérgio Tavares

São Paulo - Brasil
2025

Agradecimentos

Agradeço, primeiramente, aos meus pais, pelo apoio incondicional ao longo de toda a minha trajetória acadêmica. Seus incentivos, compreensão e esforços foram fundamentais para que eu pudesse enfrentar os desafios desta etapa e concluir este trabalho.

Aos meus amigos, deixo meu sincero agradecimento pelo companheirismo, pelas conversas, pelo apoio nos momentos de dificuldade e por tornarem essa caminhada mais leve e motivadora. A presença de vocês foi essencial durante toda a graduação.

Agradeço, de forma especial, ao meu orientador, Prof. Me. Sérgio Tavares, pela orientação, paciência, disponibilidade e contribuições técnicas ao longo do desenvolvimento deste trabalho. Seus conhecimentos e direcionamentos foram fundamentais para a construção deste projeto e para meu crescimento acadêmico e profissional.

Por fim, agradeço a todos que, direta ou indiretamente, contribuíram para a realização deste Trabalho de Conclusão de Curso.

"The challenge of machine learning is not just in creating it, but in making it better."
(Yann LeCun)

Resumo

Este Trabalho de Conclusão de Curso propõe o desenvolvimento de um sistema inteligente de detecção de intrusos (IDS) em redes de computadores, com foco em ambientes corporativos de alta demanda, como instituições financeiras. O objetivo central é combinar técnicas de aprendizado de máquina com tecnologias de processamento em tempo real, promovendo uma solução escalável, eficiente e aplicável ao mundo real.

Para isso, foi adotada uma abordagem de pesquisa quantitativa, fundamentada na experimentação controlada com dados rotulados do conjunto CICIDS2017, amplamente utilizado na literatura para simulação de tráfego de rede real, contendo registros de atividades normais e diversos tipos de ataques. Os algoritmos *XGBoost* e *LightGBM* foram escolhidos para o treinamento supervisionado devido à sua robustez, escalabilidade e bom desempenho em conjuntos de dados tabulares e desbalanceados. Foram utilizadas métricas como acurácia, precisão, *recall* e *F1-score* para avaliar os modelos, além de aspectos operacionais como latência e uso de recursos.

Visando otimizar os modelos, técnicas como *pruning* e *quantization* foram aplicadas, em conjunto com a biblioteca *ONNX Runtime*, permitindo acelerar o tempo de inferência sem comprometer a acurácia. O sistema também utiliza uma arquitetura de processamento de fluxo baseada em *Apache Kafka* e *Apache Flink*, possibilitando a análise contínua do tráfego de rede com baixa latência e alto desempenho.

Os resultados esperados envolvem a construção de um sistema IDS moderno e eficiente, capaz de detectar ataques com rapidez e precisão em ambientes de rede complexos e em tempo real. O trabalho contribui para a evolução de soluções de segurança cibernética com base em inteligência artificial e processamento distribuído, oferecendo uma abordagem promissora para proteção de sistemas críticos em larga escala.

Palavras-chaves: Detecção de Intrusos; Aprendizado de Máquina; *XGBoost*; *LightGBM*; Processamento em Tempo Real; Segurança de Redes; *ONNX Runtime*.

Lista de ilustrações

Figura 1 – Fluxo Grama da Metodologia 16
Figura 2 – Modelo TCP/IP 19
Figura 3 – Demonstração de uma Rede Local (LAN) 22
Figura 4 – Demonstração de uma Rede de Longa Distância WAN 23
Figura 5 – Demonstração das árvores *Level-Wise* e *Leaf-Wise* 31
Figura 6 – Amostragem Seletiva Baseada em Gradientes Combinada (GOSS) com Agrupamento Exclusivo de Atributos (EFB) 32
Figura 7 – Workflow do Apache Kafka 34
Figura 8 – Workflow do Apache Flink 35
Figura 9 – *Worflow* do processo PTQ e QTA 39
Figura 10 – Matriz Confusão 39
Figura 11 – Fluxo Grama 43
Figura 12 – Fluxo Grama 44
Figura 13 – Distribuição percentual da intensidade dos ataques. 45
Figura 14 – Distribuição percentual dos tipos de ataque presentes no CICIDS2017. 46
Figura 15 – Top 10 tipos de ataque por frequência absoluta. 46
Figura 16 – Comparação de métricas na melhor iteração dos modelos *XGBoost* e *LightGBM*. 47
Figura 17 – Precisão por classe de ataque entre *XGBoost* e *LightGBM*. 48
Figura 18 – Recall por classe de ataque entre *XGBoost* e *LightGBM*. 48
Figura 19 – Confiança média das previsões por tipo de ataque. 49
Figura 20 – Distribuição geral da confiança das previsões. 49
Figura 21 – Proporção entre tráfego benigno e malicioso. 50
Figura 22 – Distribuição da confiança durante o treinamento agressivo. 51
Figura 23 – Distribuição de intensidades de ataque no treinamento agressivo. 51
Figura 24 – Comparação de acurácia e *F1-score* entre modelos padrão e *ONNX Runtime*. 52
Figura 25 – Comparação de *precision* e *recall* entre modelos padrão e *ONNX Runtime*. 52

Lista de tabelas

Tabela 1 – Ferramentas que foram utilizadas 15
Tabela 2 – Configuração do Computador Utilizado 16
Tabela 3 – Diferenças entre IPv4 e IPv6 20

Lista de abreviaturas e siglas

GOSS	*Gradient-based One-Side Samplin*
EFB	*Exclusive Feature Bundlin*
AUC	Área sob a Curva
IDS	*Intrusion Detection Systems*
ML	*Machine Learning*
IoT	Internet das Coisas
GOSS	*Gradient-based One-Side Samplin*
EFB	*Exclusive Feature Bundlin*
TCP	*Transmission Control Protocol*
IP	*Internet Protocol*
IPv4	*Internet Protocol version 4Internet Protocol version 4*
IPv6	*Internet Protocol version 6*
LAN	*Local Area Networks*
WAN	Rede de Longa Distância
ATM	*Asynchronous Transfer Mode*
PPTP	*Point-to-Point Tunneling Protocol*
L2TP	*Layer 2 Tunneling Protocol*
VPN	*Virtual Private Network*
IA	Inteligência Artificial
NLP	*Natural Language Processing*
L1	*Least Absolute Shrinkage and Selection Operator*
GBDT	*Gradient Boosted Decision Trees*
GPU	*Graphics Processing Unit*
INT8	*Integer8*
FP16	*Float16*
FP32	*Float32*
ONNX	*Open Neural Network ExchangeOpen Neural Network Exchange*

PTQ	*Post-Training Quantization*
QAT	*Quantization Aware Training*
ACK	*Acknowledgment*

Sumário

1 INTRODUÇÃO . . . 11
1.1 Contexto . . . 12
1.2 Justificativa . . . 12
1.3 Objetivos . . . 13
1.3.1 Objetivo Geral . . . 13
1.3.2 Objetivos Específicos . . . 13

2 METODOLOGIA . . . 14
2.1 Tipo de Pesquisa e Abordagem . . . 14
2.2 Referencial Teórico-Metodológico . . . 15
2.3 Estudo de Caso . . . 15
2.4 Materiais e Ferramentas . . . 15
2.5 Procedimentos Metodológicos . . . 16
2.6 Considerações Finais da Metodologia . . . 17

3 REVISÃO BIBLIOGRÁFICA . . . 18
3.1 Redes de Computadores . . . 18
3.1.1 Protocolo TCP/IP . . . 18
3.1.1.1 IP – Internet Protocol . . . 19
3.1.1.2 TCP – Transmission Control Protocol . . . 20
3.1.1.3 Funcionamento Conjunto . . . 21
3.1.2 Redes Locais (LAN) . . . 21
3.1.3 Redes de Longa Distância (WAN) . . . 22
3.1.4 Redes Distribuídas . . . 24
3.1.5 Tráfego de Rede e Detecção de Intrusos . . . 24
3.1.6 Análise do Dataset CICIDS2017 . . . 24
3.1.6.1 Geração e Estrutura . . . 25
3.1.6.2 Ataques Simulados . . . 25
3.1.6.3 Camadas TCP/IP Simuladas . . . 25
3.1.6.4 Entrada dos Modelos . . . 26
3.1.6.5 Pré-processamento e Limitações . . . 26
3.2 Inteligência Artificial (IA) . . . 26
3.2.1 Algoritmos de Aprendizado de Máquina . . . 27
3.2.1.1 Principais Tipos de Aprendizado de Máquina . . . 27
3.2.2 XGBoost . . . 28
3.2.3 Pseudocódigo do XGBoost . . . 28
3.2.4 Código em Python utilizando XGBoost . . . 28
3.2.4.1 Regularização L1 e L2 . . . 29
3.2.5 LightGBM . . . 29
3.2.6 Pseudocódigo do LightGBM . . . 30
3.2.7 Código em Python utilizando LightGBM . . . 30
3.2.7.1 Level-Wise . . . 30
3.2.7.2 Leaf-Wise . . . 30
3.2.7.3 Gradient-based One-Side Sampling (GOSS) . . . 31

3.2.7.4 Exclusive Feature Bundling (EFB) . . . 32
3.2.7.5 Uso Conjunto de XGBoost e LightGBM . . . 33
3.3 Frameworks de Processamento em Tempo Real . . . 33
3.3.1 Apache Kafka . . . 34
3.3.2 Apache Flink . . . 35
3.3.3 Integração entre Kafka e Flink . . . 36
3.4 Técnicas de Otimização de Modelos . . . 36
3.4.1 ONNX . . . 37
3.4.1.1 Pruning . . . 38
3.4.1.2 Quantization . . . 38
3.4.1.3 Post-training quantization (PTQ) . . . 38
3.4.1.4 Quantization-aware training (QAT) . . . 38
3.4.2 Métricas de Avaliação de Modelos . . . 39
3.5 Trabalhos relacionados . . . 41

4 DESENVOLVIMENTO . . . 42

5 RESULTADOS E DISCUSSÃO . . . 45
5.1 Distribuição Geral dos Ataques no Conjunto CICIDS2017 . . . 45
5.2 Comparação de Métricas (Melhor Iteração) . . . 47
5.3 Análise por Classe e Confiabilidade das Previsões . . . 48
5.4 Aggressive Training e Análise de Desbalanceamento . . . 50
5.5 Análise de Desempenho ONNX Runtime . . . 52
5.6 Síntese Geral dos Resultados . . . 53
5.7 Comparação com Estudos Relacionados . . . 54

6 CONCLUSÃO . . . 55

REFERÊNCIAS . . . 57

APÊNDICES 62

APÊNDICE A – REPOSITÓRIO DO PROJETO . . . 63

APÊNDICE B – SCRIPT KAFKA PRODUCER PARA GERAÇÃO DE TRÁFEGO . . . 64

APÊNDICE C – JOB APACHE FLINK PARA DETECÇÃO DE INTRUSÕES . . . 67

APÊNDICE D – TREINAMENTO E AVALIAÇÃO DOS MODELOS 69

1 Introdução

Detectar acessos não autorizados, movimentações suspeitas ou atividades maliciosas é essencial para evitar fraudes, vazamentos de informações e interrupções nos serviços. Esse desafio se torna ainda mais urgente em ambientes financeiros, como bancos e *fintechs*, onde o tráfego de dados é intenso, constante e altamente sensível.

Diversas pesquisas têm explorado o uso de algoritmos de aprendizado de máquina para aprimorar sistemas de detecção de intruso Souza e Salles (2011), Carvalho Paulo e Silva (2015). Estudos comparativos destacam a eficácia do *XGBoost* e do *LightGBM* em tarefas de classificação. Por exemplo, Florek e Zagdański (2023) demonstraram que o *LightGBM*, com as técnicas GOSS(*Gradient-based One-Side Samplin*) e EFB (*Exclusive Feature Bundlin*), superou outras implementações em termos de velocidade e AUC (Área sob a Curva) em conjuntos de dados variados Florek e Zagdański (2023) .

Para processar grandes volumes de dados em tempo real, tecnologias de *streaming* como *Apache Kafka* e *Apache Flink* têm sido amplamente adotadas. Empresas como *PayPal* e *Capital One* utilizam essas plataformas para detectar fraudes de forma contínua, analisando o tráfego de dados à medida que ocorre One (2025), BYTEBYTEGO (2024).

No entanto, a implementação de modelos complexos de aprendizado de máquina em ambientes de produção exige otimizações para garantir desempenho e eficiência. Técnicas como *pruning* e *quantization* são utilizadas para reduzir o tamanho dos modelos e acelerar o tempo de inferência Liang et al. (2021), enquanto ferramentas como o *ONNX Runtime* da *Microsoft* permitem a otimização e execução eficiente de modelos em GPUs e também em CPUs Microsoft (2022).

1.1 Contexto

No setor financeiro, qualquer falha na detecção de intrusos pode resultar em prejuízos milionários e danos irreparáveis à reputação das empresas. Casos como o ataque ao Banco Central de Bangladesh, em 2016, que causou a perda de US$ 81 milhões BBC (2016), e a violação de dados do *JPMorgan Chase* Pan (2014), em 2014, que expôs informações de mais de 83 milhões de contas, evidenciam a gravidade do problema. Por isso, bancos, operadoras de cartão, corretoras e *fintechs* necessitam de soluções que aliem precisão, velocidade e escalabilidade.

Este projeto propõe uma solução moderna e eficiente, aplicável ao mundo real, por meio da integração de técnicas de aprendizado de máquina e arquitetura distribuída. Ao identificar padrões anômalos em tempo real e responder proativamente a ameaças, a proposta visa contribuir de forma efetiva para o avanço da segurança da informação em um dos setores mais críticos da economia.

1.2 Justificativa

Com o crescimento constante do volume de dados e da complexidade das ameaças digitais, os métodos tradicionais de segurança de rede, como *firewalls*, sistemas baseados em regras e assinaturas, já não são suficientes. É cada vez mais necessário adotar soluções inteligentes, capazes de aprender padrões de comportamento e reagir em tempo real a qualquer anomalia.

O uso dos algoritmos como *XGBoost* e *LightGBM*, ambos de código aberto, é amplamente reconhecido por seu desempenho otimizado em tarefas de classificação, especialmente quando se trata de grandes volumes de dados. O *XGBoost* se destaca por seu alto desempenho, robustez contra *overfitting*, situação em que o modelo aprende demais os dados de treinamento e perde capacidade de generalização, e por ser amplamente utilizado em competições de machine learning Chen (2015). O *LightGBM*, por sua vez, foi projetado para eficiência e escalabilidade, com suporte a aprendizado distribuído e paralelismo.

Já o *Apache Kafka* e o *Apache Flink* trazem a segurança necessária para processar esses dados de forma contínua, sem reduzir a performance Waehner (2022). As técnicas de otimização como *pruning*, *quantization* e o uso do *ONNX Runtime* tornam-se os algoritmos leves e preparados para ambientes de produção exigentes Microsoft (2022).

No setor financeiro, qualquer falha na detecção de intrusos pode significar prejuízos milionários e danos irreparáveis à reputação das empresas. Por isso, bancos, operadoras de cartão, corretoras e *fintechs* precisam de soluções que unam precisão, velocidade e escalabilidade.

1.3 Objetivos

Os objetivos deste trabalho estão divididos em objetivo geral, que se propõe a responder à questão de pesquisa, e objetivos específicos, que detalham os passos necessários para se atingir o objetivo geral.

1.3.1 Objetivo Geral

Este trabalho tem como objetivo propor e desenvolver um sistema inteligente de detecção de intrusos em redes de computadores, com foco em operação em tempo real e alta eficiência computacional. Para isso, serão exploradas técnicas de aprendizado de máquina e processamento de dados em fluxo contínuo, que serão aplicadas a contextos com grande volume de tráfego e demanda por respostas rápidas.

1.3.2 Objetivos Específicos

- Avaliar os algoritimos *XGBoost* e *LightGBM* com relação as suas vantagens e limitações no contexto da detecção de intrusos.
- Avaliar as técnicas de otimização de *pruning*, *quantization* e uso do *ONNX Runtime*, a fim de verificar se os modelos se tornam mais leves, rápidos e eficientes para execução em tempo real.
- Avaliar o sistema quanto à sua precisão, velocidade de resposta (latência) e consumo de recursos computacionais.
- Testar e validar a solução utilizando o conjunto de dados CICIDS2017, que simula tráfego de rede realista com diferentes tipos de ataques.

2 Metodologia

De acordo com Bruyne, Herman e Schoutheete (1977), a metodologia representa o conjunto de caminhos e estratégias utilizados para alcançar os objetivos de uma pesquisa, orientando o pesquisador na escolha dos métodos, técnicas e procedimentos que possibilitam a obtenção e análise dos resultados. Este capítulo descreve os métodos empregados no desenvolvimento do presente trabalho, cujo foco é a otimização de modelos de aprendizado de máquina aplicados à detecção de intrusões em redes de computadores, com ênfase na eficiência computacional e na inferência em tempo real.

A metodologia adota o paradigma da *Design Science Research (DSR)*, apropriado para pesquisas que envolvem o desenvolvimento de artefatos computacionais como meio de geração de conhecimento científico. Nesse contexto, o artefato proposto consiste em um *pipeline* otimizado para detecção de intrusões, construído a partir de modelos de aprendizado supervisionado (*XGBoost, LightGBM* e *Random Forest*), exportados para o formato *ONNX* e integrados a um ambiente de processamento de fluxo baseado em *Apache Kafka* e *Apache Flink*. O objetivo central é demonstrar, por meio de experimentação prática, que as técnicas de otimização aplicadas (como *pruning*, *quantization* e o uso do *ONNX Runtime*) proporcionam ganhos reais de desempenho e escalabilidade.

2.1 Tipo de Pesquisa e Abordagem

Segundo Prodanov e Freitas (2013), a pesquisa científica pode ser classificada quanto à sua natureza, objetivos e abordagem. Este trabalho caracteriza-se como uma pesquisa aplicada, pois visa oferecer uma solução prática a um problema real: a detecção eficiente de intrusões em redes com baixa latência. A abordagem adotada é quantitativa e experimental, envolvendo a coleta e análise de dados empíricos relacionados ao desempenho dos modelos, com base em métricas objetivas como acurácia, precisão, *recall*, *F1-score* e tempo médio de inferência. Elementos qualitativos também foram considerados na interpretação e comparação dos resultados das técnicas de otimização.

A estratégia metodológica fundamenta-se na *Design Science Research* (DSR), conforme o modelo de processo proposto por Peffers et al. (2007), que estrutura o desenvolvimento e avaliação de artefatos tecnológicos em seis etapas:

(1) identificação do problema e motivação; (2) definição dos objetivos da solução; (3) projeto e desenvolvimento do artefato; (4) demonstração; (5) avaliação; e (6) comunicação dos resultados.

2.2 Referencial Teórico-Metodológico

A primeira etapa consistiu em uma revisão bibliográfica sistemática abordando fundamentos de aprendizado de máquina, detecção de intrusões, otimização de modelos e processamento de dados em fluxo. O levantamento teórico foi realizado com base em livros, artigos científicos e trabalhos acadêmicos obtidos em bases de dados como *IEEE Xplore, Scopus, ScienceDirect* e*Google Scholar*, além de periódicos nacionais indexados.

Essa etapa permitiu identificar as técnicas mais eficazes em sistemas de detecção de intrusão (IDS), os modelos com melhor desempenho e os métodos de otimização aplicáveis à inferência em tempo real, como *pruning*, *quantization* e *ONNX Runtime*. A partir dessas referências, definiu-se o conjunto de ferramentas e algoritmos utilizados no desenvolvimento do artefato.

2.3 Estudo de Caso

O estudo de caso adotou o conjunto de dados CICIDS2017, amplamente reconhecido por representar tráfego de rede realista e rotulado com diferentes tipos de ataques cibernéticos. O objetivo foi avaliar a eficácia das técnicas de otimização de modelos na detecção de intrusões em fluxo contínuo de dados.

O ambiente experimental foi configurado localmente, simulando uma arquitetura de *stream processing* em tempo real. O *Apache Kafka* foi utilizado para ingestão e distribuição das amostras, enquanto o *Apache Flink* executou o processamento do fluxo e as inferências. O módulo de decisão foi implementado com o *ONNX Runtime*, responsável pela inferência dos modelos exportados em formato *ONNX*. Essa integração resultou em um pipeline funcional e escalável, capaz de simular o comportamento de um IDS corporativo.

2.4 Materiais e Ferramentas

As principais ferramentas e recursos utilizados foram:

Tabela 1 – Ferramentas que foram utilizadas

Categoria	**Ferramenta/Especificação**
Linguagem de programação	*Python* 3.11.9
Bibliotecas de aprendizado de máquina	*Scikit-Learn, XGBoost, LightGBM*
Framework de otimização e inferência	*ONNX* e *ONNX Runtime*
Frameworks de fluxo de dados	*Apache Kafka* e *Apache Flink*
Técnicas de otimização	*pruning* e *quantization* (FP16 e INT8)
Sistema operacional	*Windows* 11
Versão do sistema operacional	24H2

Fonte: Elaborado pelo autor

Tabela 2 – Configuração do Computador Utilizado

Componente	**Especificação**
Sistema Operacional	*Windows* 11
Processador	AMD Ryzen 7 5800X3D
Modo da BIOS	UEFI
Tipo do Sistema	PC baseado em x64
Placa-Mãe (*BaseBoard*)	ASUSTek COMPUTER INC.
Memória RAM	32 GB
HD (Disco Rígido)	4 TB
SSD (Disco Sólido)	1 TB
Placa de Vídeo (GPU)	NVIDIA GeForce RTX 4060
Versão do sistema operacional	24H2

Fonte: Elaborado pelo autor

2.5 Procedimentos Metodológicos

A metodologia prática foi estruturada em etapas sequenciais e iterativas, conforme o modelo *Design Science Research*:

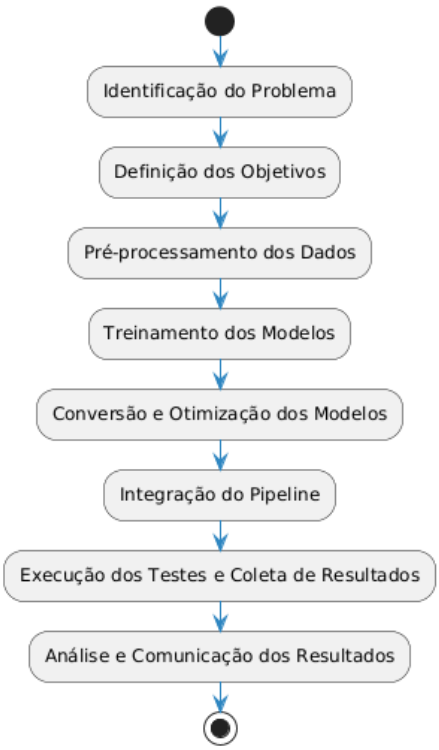


Figura 1 – Fluxo Grama da Metodologia

Fonte: Elaborado pelo autor

O desenvolvimento deste trabalho foi estruturado em diferentes etapas metodológicas, que juntas compõem o processo completo de construção e avaliação do sistema proposto. Inicialmente, realizou-se a identificação do problema, que consistiu na análise dos desafios inerentes à detecção de intrusões em tempo real, com ênfase no alto custo computacional envolvido na inferência de modelos complexos.

Em seguida, passou-se à definição dos objetivos, centrada no desenvolvimento de um *pipeline* otimizado capaz de reduzir a latência e o consumo de recursos computacionais, preservando ao mesmo tempo um alto desempenho preditivo.

Na etapa de pré-processamento dos dados, foram aplicadas técnicas de limpeza e normalização sobre o conjunto CICIDS2017, incluindo a codificação de variáveis categóricas e o balanceamento das classes, a fim de minimizar o impacto do desbalanceamento de amostras sobre o treinamento dos modelos.

Posteriormente, realizou-se o treinamento dos modelos, abrangendo a implementação e o ajuste de hiperparâmetros para os algoritmos *XGBoost, LightGBM* e *Random Forest*. Após o treinamento, foi conduzida a fase de conversão e otimização, na qual os modelos foram exportados para o formato *ONNX* e submetidos a técnicas de *pruning*, *quantizatione* otimizações automáticas de grafo por meio do *ONNX Runtime*.

Com os modelos otimizados, procedeu-se à integração do pipeline, envolvendo a configuração dos serviços *Apache Kafka* e *Apache Flink* para ingestão e processamento contínuo dos dados, permitindo a execução em fluxo dos modelos otimizados no ambiente de inferência *ONNX Runtime*.

Na sequência, ocorreu a execução dos testes e coleta de resultados, etapa em que se avaliou empiricamente o desempenho do sistema por meio de métricas como acurácia, precisão, *recall*, *F1-score*, latência média, *throughput* e consumo de CPU e memória.

Por fim, foi conduzida a análise e comunicação dos resultados, com a discussão detalhada dos impactos das otimizações sobre o desempenho dos modelos e a elaboração das conclusões do estudo, consolidando as contribuições alcançadas.

2.6 Considerações Finais da Metodologia

A metodologia combinou o rigor teórico da *Design Science Research* com a experimentação prática em aprendizado de máquina e otimização computacional. Esse processo assegurou a reprodutibilidade dos resultados e contribuiu tanto para a solução prática da detecção de intrusões em tempo real quanto para o avanço do conhecimento científico nas áreas de sistemas inteligentes e segurança da informação

3 Revisão Bibliográfica

3.1 Redes de Computadores

Redes de computadores são sistemas interconectados que permitem a troca de dados entre dispositivos por meio de protocolos de comunicação. Elas são fundamentais para a operação de sistemas distribuídos, serviços em nuvem, dispositivos IoT e, especialmente, para ambientes que exigem monitoramento de segurança em tempo real Tanenbaum e Wetherall (2011).

Segundo Tanenbaum e Wetherall (2011), redes podem ser classificadas conforme seu alcance geográfico, estrutura e propósito. Entre os principais tipos destacam-se as redes locais (*LANs, Local Area Networks*) e as redes distribuídas, que exercem papel central em ambientes empresariais e aplicações de segurança digital.

Como destaca Kurose e Ross (2021), as *LANs* geralmente operam com tecnologias *Ethernet* e protocolos como o TCP/IP, permitindo a coleta sistemática de dados de rede com precisão temporal, requisito essencial para a atuação de sistemas de *Intrusion Detection Systems (IDS)* baseados em aprendizado de máquina.

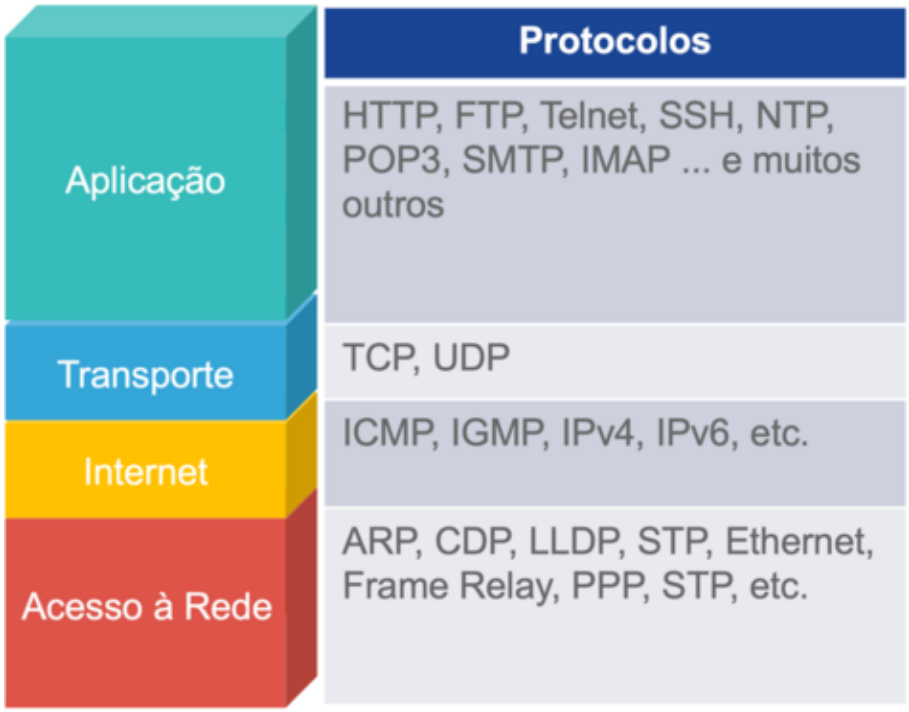
3.1.1 Protocolo TCP/IP

A comunicação em redes de computadores é possibilitada por um conjunto de regras e padrões chamados protocolos. Esses protocolos definem como os dados são organizados, transmitidos, roteados e recebidos entre dispositivos conectados. Entre os mais fundamentais estão o TCP *(Transmission Control Protocol)* e o IP *(Internet Protocol)*, que juntos formam a base da arquitetura da Internet moderna, conhecida como pilha de protocolos TCP/IP Kurose e Ross (2021).

O protocolo IP é responsável por endereçar e encaminhar pacotes de dados entre diferentes dispositivos em uma rede. Ele garante que os dados alcancem o destino correto por meio de endereços IP únicos atribuídos a cada máquina. Por outro lado, o TCP complementa o IP ao oferecer confiabilidade na comunicação: ele assegura que os pacotes enviados cheguem ao destino na ordem correta, sem perdas ou duplicações, por meio de técnicas como confirmação de recebimento ACK (*Acknowledgment*), retransmissões e controle de fluxo.

Essa separação de responsabilidades permite que a transmissão de dados seja tanto eficiente quanto confiável, sendo especialmente importante em aplicações críticas como segurança de rede, onde a integridade e a ordem dos dados são essenciais para detectar atividades suspeitas com precisão Kurose e Ross (2021).

Figura 2 – Modelo TCP/IP



Fonte: Forum (2020)

3.1.1.1 IP – Internet Protocol

O IP é responsável por endereçar e encaminhar pacotes de dados entre os dispositivos em uma rede. Cada dispositivo conectado recebe um endereço IP único, que funciona como um identificador de origem ou destino, semelhante ao endereço de uma casa em um sistema de correios Tanenbaum e Wetherall (2011), AWS (2022).

O protocolo IP atua na camada de rede do modelo OSI e trabalha com a ideia de datagramas, que são pacotes de dados enviados de forma independente, sem garantia de entrega, ordem ou integridade. Por si só, o IP é um protocolo não confiável, mas extremamente eficiente para entregar pacotes em redes interconectadas Tanenbaum e Wetherall (2011), AWS (2022).

Existem atualmente duas versões principais do protocolo IP em uso: o IPv4 (*Internet Protocol version 4*), que utiliza endereços de 32 bits, e o IPv6 (Internet Protocol version 6), que emprega endereços de 128 bits. O IPv6 foi criado para resolver a limitação de endereços disponíveis no IPv4, além de introduzir melhorias em segurança, roteamento e desempenho. A seguir, apresenta-se uma comparação entre essas duas versões do protocolo IP Tanenbaum e Wetherall (2011), AWS (2022).

Tabela 3 – Diferenças entre IPv4 e IPv6

Característica	**IPv4**	**IPv6**
Comprimento do endereço	32 *bits*	128 *bits*
Formato do endereço	Decimal com 4 blocos separados por pontos (ex: 192.168.0.1)	Hexadecimal com 8 blocos separados por dois-pontos (ex: 2 001:0db8:85a3::8a2e:0370:733 4)
Quantidade de endereços possíveis	Aproximadamente 4,3 bilhões	Aproximadamente $3,4 \times 10^{38}$ (quase ilimitado)
Configuração de endereço	Manual ou via DHCP	Autoconfiguração sem estado (SLAAC), DHCPv6 ou configuração manual
Fragmentação de pacotes	Realizada pelos roteadores	Realizada apenas pelo *host* de origem
Cabeçalho do pacote	Simples, mas com opções variáveis e tamanho variável (20–60 *bytes*)	Mais eficiente, com cabeçalho fixo de 40 bytes e extensões opcionais
Soma de verificação do cabeçalho	Presente (verificação de erro no cabeçalho)	Ausente (confia em protocolos superiores como TCP/UDP)
Suporte a comunicação *Anycast*	Não nativo	Suportado nativamente
Mecanismo de tradução (NAT)	Amplamente utilizado para economizar endereços	Desnecessário, graças ao grande número de IPs disponíveis
Segurança	Opcional (IPSec é adicional)	Integrado ao protocolo (IPSec obrigatório)
Endereços privados	Usados internamente (ex: 10.x.x.x, 172.16.x.x - 172.31.x.x, 192.168.x.x)	Não há necessidade de endereços privados devido à quantidade quase infinita de IPs

Fonte: Adaptado de Tanenbaum e Wetherall (2011) e AWS (2022).

3.1.1.2 TCP – Transmission Control Protocol

O Protocolo de Controle de Transmissão (TCP) é um dos pilares da Internet, operando na camada de transporte em conjunto com o Protocolo da Internet (IP). Ele oferece um serviço de comunicação lógica e de transferência de dados confiável e orientado à conexão entre aplicações em diferentes hospedeiros. É importante notar que o TCP não garante vazão ou temporização às aplicações Paes (2023), Kurose e Ross (2021).

A conexão TCP é estabelecida através de uma "apresentação de três vias"(*three-way handshake*) e é *full-duplex*, permitindo que ambos os lados enviem e recebam dados simultaneamente Paes (2023), Kurose e Ross (2021). Os dados são organizados em segmentos, que incluem um cabeçalho (tipicamente 20 *bytes*) e um campo de dados, cujo tamanho máximo é definido pelo MSS (*Maximum Segment Size*) Kurose e Ross (2021).

Para garantir a entrega confiável dos dados, o TCP utiliza mecanismos como números de sequência, reconhecimentos cumulativos (ACKs) e temporizadores de retransmissão. Se um segmento não é reconhecido a tempo ou se três ACKs duplicados são recebidos, o segmento é retransmitido. O receptor do TCP geralmente armazena os segmentos que chegam fora de ordem Kurose e Ross (2021).

Além disso, o TCP implementa controle de fluxo para evitar que o remetente sobrecarregue o receptor, e controle de congestionamento fim a fim . A perda de segmentos é interpretada como um sinal de congestionamento na rede, levando o TCP a reduzir sua taxa de envio . O algoritmo de controle de congestionamento aumenta aditivamente sua taxa e a reduz multiplicativamente em caso de perda de pacotes. O estabelecimento da conexão e a "partida lenta"(*slow start*) podem adicionar um atraso significativo Kurose e Ross (2021).

Quanto à segurança, o TCP e o UDP não oferecem criptografia por si só. No entanto, tecnologias como a Segurança na Camada de Transporte (TLS), antigamente conhecida como SSL, podem ser usadas sobre o TCP para fornecer serviços de segurança como criptografia, integridade dos dados e autenticação de ponto final Kurose e Ross (2021).

3.1.1.3 Funcionamento Conjunto

O modelo TCP/IP funciona de forma integrada: enquanto o IP encaminha os pacotes entre dispositivos, o TCP garante que esses pacotes sejam entregues com integridade e na ordem certa. Essa colaboração entre os dois protocolos é a base do funcionamento da Internet e de muitas redes corporativas Kurose e Ross (2021), Tanenbaum e Wetherall (2011).

3.1.2 Redes Locais (LAN)

Uma LAN é uma rede que cobre uma área geograficamente limitada, como um escritório, campus universitário ou data center. Ela oferece alta taxa de transmissão e baixa latência, sendo ideal para comunicação entre as estações de trabalho, servidores e dispositivos periféricos Systems (2023). As LANs são frequentemente utilizadas como ambiente para análises e monitoramento de tráfego em soluções de detecção de intrusos, uma vez que capturam de forma eficaz os pacotes de dados trocados internamente.

As principais características das LANs incluem a alta velocidade, proporcionada pela proximidade física dos dispositivos, o que permite elevadas taxas de transferência de dados; a baixa latência, que resulta em tempos de resposta rápidos na comunicação entre os elementos da rede; e o gerenciamento simplificado, uma vez que, por estarem confinadas a um espaço físico limitado, essas redes são mais fáceis de administrar e solucionar eventuais problemas.

Figura 3 – Demonstração de uma Rede Local (LAN)

Fonte: Sigma (2022)

3.1.3 Redes de Longa Distância (WAN)

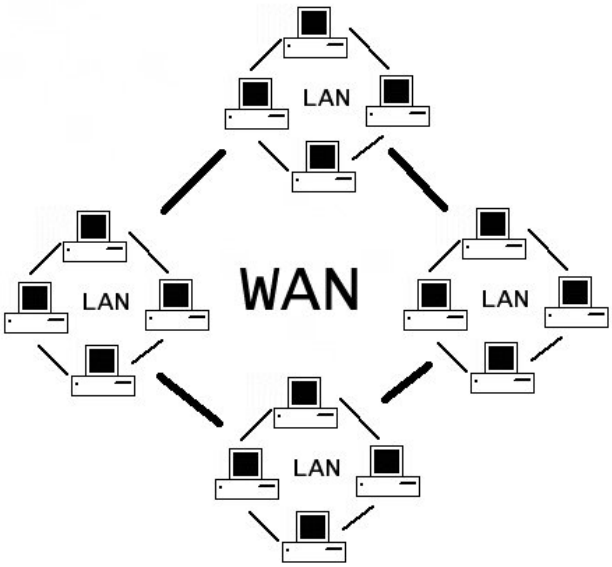
Uma Rede de Longa Distância (WAN) é uma infraestrutura de comunicação que conecta redes locais (LANs) separadas por grandes distâncias geográficas, permitindo que dispositivos em diferentes localidades se comuniquem como se estivessem na mesma rede. As WANs abrangem regiões extensas, podendo interligar cidades, países ou até mesmo continentes. A *Internet* é o exemplo mais notável de uma WAN, conectando redes em todo o mundo e permitindo a troca global de informações IBM (2023).

As principais características das WANs incluem a cobertura geográfica ampla, com capacidade para abranger vastas regiões; a interconexão de LANs, permitindo que diversas redes locais se comuniquem entre si como uma única rede integrada; e o uso de infraestruturas diversas, como linhas alugadas, conexões via satélite e enlaces de fibra óptica. Essas características tornam as WANs indispensáveis para organizações com múltiplas filiais distribuídas geograficamente, pois possibilitam a comunicação eficiente e o compartilhamento de recursos entre diferentes unidades.

De acordo com a IBM (2023) existem diferentes tipos de WANs, sendo as mais comuns a WAN Comutada (*Switched WAN*) e a WAN Ponto a Ponto (*Point-to-Point WAN*). A WAN comutada utiliza uma infraestrutura compartilhada, geralmente fornecida por operadoras de telecomunicação, onde os dados são roteados por meio de um *hub* central conhecido como *switching exchange*. Essa configuração costuma empregar tecnologias como o ATM (*Asynchronous Transfer Mode*), que transmite os dados em pacotes de tamanho fixo, sendo especialmente indicada para aplicações que exigem transmissão rápida e confiável, como as que operam em tempo real.

Já a WAN ponto a ponto estabelece uma conexão direta entre dois pontos por meio de uma linha dedicada (*leased line*), oferecendo uma comunicação privada e segura entre localidades. Esse tipo de rede frequentemente utiliza protocolos como o PPTP (*Point-to-Point Tunneling Protocol*) e o L2TP (*Layer 2 Tunneling Protocol*) para criar túneis de comunicação seguros, como no caso das redes VPN. É uma escolha comum em ambientes onde a segurança, a estabilidade e a velocidade de conexão são fatores críticos.

Figura 4 – Demonstração de uma Rede de Longa Distância WAN



Fonte: Sigma (2022)

3.1.4 Redes Distribuídas

Redes distribuídas referem-se a infraestruturas compostas por múltiplos sistemas conectados, muitas vezes geograficamente dispersos, que colaboram para executar tarefas de forma coordenada. Em ambientes distribuídos, como sistemas bancários ou plataformas de computação em nuvem, o tráfego de rede se torna mais complexo e volumoso, aumentando a necessidade de soluções de segurança adaptativas e escaláveis.

Para sistemas de detecção de intrusos, esse tipo de rede impõe desafios adicionais, como a latência na sincronização de dados, balanceamento de carga e necessidade de consistência entre os nós. Segundo Coulouris et al. (2012), redes distribuídas exigem protocolos robustos de comunicação e mecanismos de monitoramento contínuo para garantir integridade, disponibilidade e confidencialidade das informações.

Desafios das Redes Distribuídas:

- Latência na Sincronização de Dados: Garantir que todos os nós da rede tenham dados atualizados simultaneamente pode ser desafiador.
- Balanceamento de Carga: Distribuir eficientemente as tarefas entre os nós para evitar sobrecarga em partes específicas da rede.
- Consistência e Integridade dos Dados: Assegurar que os dados permaneçam consistentes e íntegros em todos os nós da rede.

Para enfrentar esses desafios, são necessários protocolos robustos de comunicação e mecanismos de monitoramento contínuo que garantam a integridade, disponibilidade e confidencialidade das informações.

3.1.5 Tráfego de Rede e Detecção de Intrusos

Tanto em redes locais quanto em distribuídas, o tráfego de rede pode conter padrões que indicam comportamentos anômalos, como escaneamento de portas, tentativas de login forçado ou movimentações laterais não autorizadas. A captura e análise desse tráfego são essenciais para que sistemas baseados em aprendizado de máquina possam identificar ataques com precisão.

Conjuntos de dados amplamente utilizados, como o CICIDS2017, simulam tráfego de rede realista em ambientes controlados de *LANs* e *WANs*, possibilitando o treinamento e avaliação de modelos supervisionados de detecção de intrusos. Portanto, a compreensão da arquitetura das redes onde esses modelos serão aplicados é crucial para o sucesso da implementação de sistemas IDS inteligentes Canadian Institute for Cybersecurity (2017).

3.1.6 Análise do Dataset CICIDS2017

O dataset CICIDS2017 é amplamente utilizado na pesquisa sobre Sistemas de Detecção de Intrusos em Redes (NID). Desenvolvido pelo *Canadian Institute for Cybersecurity* (CIC), ele visa superar limitações de conjuntos anteriores ao oferecer um ambiente mais realista e representativo de tráfego de rede, simulando tanto comportamentos benignos quanto maliciosos Canadian Institute for Cybersecurity (2017).

O CICIDS2017 foi concebido como plataforma robusta para:

- Desenvolvimento e validação de modelos baseados em aprendizado de máquina (ML) e aprendizado profundo (DL);
- Classificação binária e multiclasse de tráfego de rede;
- Estudo sobre desequilíbrio de classes;
- Seleção de atributos e análise de desempenho computacional Soumik (2024).

3.1.6.1 Geração e Estrutura

Segundo os autores do dataset CICIDS2017, a base foi construída em um ambiente de rede controlado, contendo *switches*, roteadores, *firewalls* e máquinas com diferentes sistemas operacionais (*Windows*, *Ubuntu* e *Mac OS*). A coleta de dados foi realizada ao longo de cinco dias úteis (de 3 a 7 de julho de 2017), sendo a segunda-feira dedicada apenas a tráfego legítimo, enquanto os demais dias incluíram ataques simulados Smith e Lee (2024).

Os dados foram disponibilizados em dois formatos: arquivos PCAP, contendo a captura completa dos pacotes, e arquivos CSV, que resumem os fluxos de rede com atributos extraídos automaticamente pela ferramenta *CICFlowMeter*. Cada fluxo inclui 83 atributos estatísticos. Contudo, para experimentos de aprendizado de máquina, geralmente são utilizados 77 atributos, com a exclusão de campos como IPs, portas e *timestamp*, para evitar viés na aprendizagem dos modelos Engelen (2021).

3.1.6.2 Ataques Simulados

O CICIDS2017 contempla uma ampla variedade de ataques, incluindo:

- Força bruta (*FTP-Patator, SSH-Patator*);
- DoS e DDoS (*Hulk, GoldenEye, Slowloris, LOIC*);
- Web attacks (*Brute Force, SQL Injection, XSS*);
- Infiltração e ataques de *Botnet*;
- *Port Scan* e *Heartbleed* Soumik (2024).

3.1.6.3 Camadas TCP/IP Simuladas

O *dataset* fornece atributos que permitem inferência sobre diferentes camadas:

- Camada de Rede (3): IPs de origem e destino;
- Camada de Transporte (4): Portas, protocolo (TCP/UDP), *flags* TCP;
- Camada de Aplicação (7): Protocolos como HTTP, FTP, SSH, inferidos a partir de estatísticas de *payload* e fluxo TÜBİTAK (2023).

3.1.6.4 Entrada dos Modelos

Cada instância no *dataset* representa um fluxo bidirecional de comunicação. Os modelos recebem como entrada os 77 atributos numéricos extraídos, com o rótulo correspondente a uma das 15 classes (benigno ou um tipo específico de ataque).

3.1.6.5 Pré-processamento e Limitações

Antes do uso, é comum aplicar:

- Remoção de valores nulos, normalização dos atributos;
- Balanceamento com SMOTE e *undersampling*;
- Seleção de atributos com base em importância e correlação Anonymous (2023).

Estudos recentes identificaram falhas na rotulagem e na interpretação dos fluxos TCP (e.g. fluxos apêndices, ignorância de pacotes RST), levando a versões corrigidas do *dataset* e melhores *benchmarks* Engelen (2021).

3.2 Inteligência Artificial (IA)

A Inteligência Artificial (IA) é um campo da ciência da computação que tem como objetivo desenvolver sistemas e algoritmos capazes de executar tarefas que normalmente exigiriam inteligência humana, como reconhecimento de padrões, tomada de decisões, aprendizado e adaptação. Em outras palavras, a IA busca simular a capacidade humana de raciocinar, perceber, aprender e resolver problemas em ambientes variados Russell e Norvig (2010).

Os sistemas de IA podem ser divididos em diferentes categorias, dependendo do seu grau de autonomia e capacidade de aprendizado. No contexto atual, os modelos mais utilizados pertencem à chamada IA fraca ou estreita, que são desenvolvidos para realizar tarefas específicas (como reconhecimento facial, recomendações de produtos ou detecção de intrusos em redes). Já a IA forte, ainda em estágio teórico, se refere a sistemas com capacidade cognitiva comparável à humana, capazes de compreender e executar qualquer tarefa intelectual Autentify (2024).

Um dos maiores avanços da IA nas últimas décadas foi o surgimento do Aprendizado de Máquina(*Machine Learning*), que permite que os algoritmos aprendam automaticamente a partir de dados, sem serem explicitamente programados para cada tarefa. Dentro da IA, também se destacam áreas como visão computacional, processamento de linguagem natural (NLP, *Natural Language Processing*) e Aprendizado Profundo (*Deep Learning*), que permitem aplicações mais sofisticadas e precisas em diferentes setores, como saúde, finanças, segurança e telecomunicações.

No caso da detecção de intrusos, a IA tem papel central ao possibilitar que sistemas reconheçam comportamentos anômalos em tempo real, adaptando-se a novas ameaças sem depender exclusivamente de regras fixas ou assinaturas previamente definidas.

3.2.1 Algoritmos de Aprendizado de Máquina

O aprendizado de máquina (ML, Learning) é uma subárea da inteligência artificial que se concentra no desenvolvimento de algoritmos capazes de aprender padrões a partir de dados e realizar previsões ou decisões com base nesse aprendizado. Diferente da programação tradicional, em que regras são explicitamente codificadas por humanos, no ML os modelos são treinados com exemplos, permitindo que eles generalizem e façam inferências sobre novos dados.

Essa abordagem tem se mostrado particularmente eficiente em cenários complexos, como reconhecimento de imagem, processamento de linguagem natural, recomendação de conteúdo e, especialmente, em segurança cibernética. Em sistemas de detecção de intrusos (IDS, *Intrusion Detection Systems*), o aprendizado de máquina é capaz de identificar comportamentos anômalos sem depender exclusivamente de assinaturas ou regras fixas, tornando-o eficaz contra ataques desconhecidos ou variantes não mapeadas previamente Pacheco (2021).

3.2.1.1 Principais Tipos de Aprendizado de Máquina

Os algoritmos de aprendizado de máquina são tradicionalmente agrupados em três categorias principais, de acordo com a forma que lidam com os dados.

O aprendizado supervisionado utiliza conjuntos de dados rotulados, ou seja, exemplos de entrada com as saídas desejadas já conhecidas. O modelo aprende a mapear essas entradas para as saídas corretas. É amplamente utilizado em tarefas de classificação (ex: detectar se um pacote é malicioso ou não) e regressão (ex: prever a quantidade de tráfego futuro).

Já o aprendizado não supervisionado ele trabalha com dados não rotulados. O objetivo é identificar estruturas ou padrões ocultos, como agrupamentos (*clusters*) ou regras de associação. Essa abordagem é útil para descobrir perfis de comportamento ou detectar anomalias sem conhecimento prévio.

E o aprendizado por reforço baseia-se na interação entre um agente que aprende interagindo com um ambiente. Ele toma decisões e recebe recompensas (ou penalidades), ajustando sua estratégia para maximizar a recompensa acumulada. É aplicado em jogos, controle robótico e segurança adaptativa.

Essas abordagens tornam o aprendizado de máquina altamente flexível, permitindo sua aplicação em cenários de segurança, como detecção de intrusos em redes corporativas, onde novos padrões de ataque podem surgir frequentemente e a capacidade de adaptação é essencial Ferreira (2017).

3.2.2 XGBoost

O XGBoost (*Extreme Gradient Boosting*) é uma biblioteca de aprendizado de máquina baseada em árvores de decisão que utiliza a técnica de *boosting* de gradiente para melhorar o desempenho preditivo de modelos sequenciais. Por ser um algoritmo de aprendizado supervisionado, ele exige um conjunto de dados rotulado durante o treinamento, ou seja, um conjunto no qual a saída esperada é conhecida, para que possa aprender a identificar padrões e fazer previsões em novos dados. Ele se tornou uma das ferramentas mais populares em competições de ciência de dados, como as do *Kaggle*, devido à sua eficiência, precisão e capacidade de lidar com dados tabulares complexos, conforme Pinheiro (2023).

Uma das principais vantagens do *XGBoost* está em sua robustez contra *overfitting*, alcançada por meio de regularização L1 (lasso) e L2 (*ridge*), além de estratégias como *pruning* e controle de profundidade das árvores. A biblioteca também suporta execução paralela, cache inteligente de dados e manejo eficiente de valores ausentes, o que a torna altamente escalável para grandes volumes de dados. Conforme apontado pela Chen (2015), o *XGBoost* continua sendo amplamente adotado em soluções reais de aprendizado de máquina, sendo especialmente indicado quando há a necessidade de alta acurácia e controle fino sobre o modelo, como pode ser visto no pseudocódigo 1.

3.2.3 Pseudocódigo do XGBoost

Algorithm 1 Pseudocódigo simplificado do *XGBoost*

Require: Conjunto de dados $D = \{(x_1, y_1), ..., (x_n, y_n)\}$, número de iterações K

1: Inicialize o modelo: $f_0(x) = 0$
2: **for** $k = 1$ **até** K **do**
3: Calcular os resíduos: $g_i = \frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}$
4: Ajustar uma árvore $h_k(x)$ aos resíduos g_i
5: Atualizar o modelo: $f_k(x) = f_{k-1}(x) + \eta \cdot h_k(x)$
6: Aplicar regularização (L1 ou L2)
7: **end for**
8: **return** Modelo final $f_K(x)$

3.2.4 Código em Python utilizando XGBoost

```
import xgboost as xgb

model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

3.2.4.1 Regularização L1 e L2

Como havia dito antes as principais vantagens do *XGBoost* está em sua robustez contra *overfitting*, alcançada por meio da aplicação de técnicas de regularização. Entre elas, destacam-se a regularização **L1** e **L2**, ambas adicionadas à função de perda com o objetivo de controlar a complexidade do modelo e reduzir a variância.

A regularização L1, também conhecida como Lasso (*Least Absolute Shrinkage and Selection Operator*), adiciona à função de perda a soma dos valores absolutos dos coeficientes dos parâmetros, conforme representado na Equação 3.1:

$$\text{Loss}_{L1} = \text{Erro} + \lambda \sum_{i=1}^{n} |w_i| \tag{3.1}$$

Essa penalização promove a redução de alguns coeficientes a zero, realizando uma seleção automática de atributos. Como resultado, o modelo se torna mais simples e interpretável, favorecendo a eliminação de variáveis irrelevantes.

Já a regularização L2, conhecida como *Ridge*, adiciona à função de perda a soma dos quadrados dos coeficientes, como mostrado na Equação 3.2:

$$\text{Loss}_{L2} = \text{Erro} + \lambda \sum_{i=1}^{n} w_i^2 \tag{3.2}$$

Diferentemente da L1, essa técnica não zera os coeficientes, mas os reduz em magnitude. Isso contribui para a estabilidade do modelo e sua capacidade de generalização, especialmente especialmente em situações de multicolinearidade, quando variáveis independentes estão altamente correlacionadas entre si, o que pode gerar instabilidade nos coeficientes e prejudicar a interpretação e robustez do modelo.

O *XGBoost* combina ambas as penalizações (L1 + L2), o que permite alcançar um equilíbrio entre simplicidade e desempenho preditivo, sendo especialmente eficaz em conjuntos de dados de alta dimensionalidade.

3.2.5 LightGBM

O *LightGBM* (*Light Gradient Boosting Machine*) é um framework de aprendizado de máquina desenvolvido pela *Microsoft*, baseado na técnica de *Gradient Boosted Decision Trees* (GBDT) pertencente à classe dos modelos supervisionados. Ele foi projetado com foco em desempenho, escalabilidade e eficiência de memória, sendo ideal para aplicações em larga escala e ambientes que demandam alto volume de dados, como sistemas de detecção de intrusos, motores de recomendação e análise de fraudes em tempo real.

Um dos principais diferenciais do *LightGBM* em relação a outras bibliotecas de *boosting*, como o *XGBoost*, está na forma como ele organiza e processa os dados durante o treinamento. Ele foi projetado para treinar modelos com alta velocidade e baixo uso de memória, mesmo em bases de dados de grande dimensão ou com muitas variáveis dispersas. Para isso, o *framework* utiliza estruturas de dados otimizadas e algoritmos que reduzem a complexidade computacional sem comprometer a acurácia.

Além disso, o *LightGBM* adota um modelo de crescimento de árvore do tipo *leaf-wise*, que prioriza a divisão dos nós com maior potencial de redução de erro, resultando em modelos mais precisos e com convergência mais rápida. Apesar de esse método poder aumentar o risco de *overfitting* em certos cenários, sua combinação com estratégias robustas de regularização permite obter excelente desempenho preditivo, especialmente em tarefas com grandes quantidades de dados e variáveis Ke et al. (2017).

3.2.6 Pseudocódigo do LightGBM

Algorithm 2 Pseudocódigo simplificado do *LightGBM*

Require: Conjunto de dados D, número de árvores K

Aplicar EFB (*Exclusive Feature Bundling*) para reduzir a dimensionalidade
for $k = 1$ **até** K **do**
    Calcular os gradientes dos exemplos
    Aplicar GOSS (*Gradient-based One-Side Sampling*):
        Manter os exemplos com maiores gradientes
        Amostrar aleatoriamente os exemplos restantes
    Construir uma árvore *leaf-wise* com os exemplos amostrados
    Atualizar o modelo com a nova árvore
end for
return Modelo final com K árvores otimizadas

3.2.7 Código em Python utilizando LightGBM

```python
import lightgbm as lgb

model = lgb.LGBMClassifier(n_estimators=100, learning_rate=0.1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

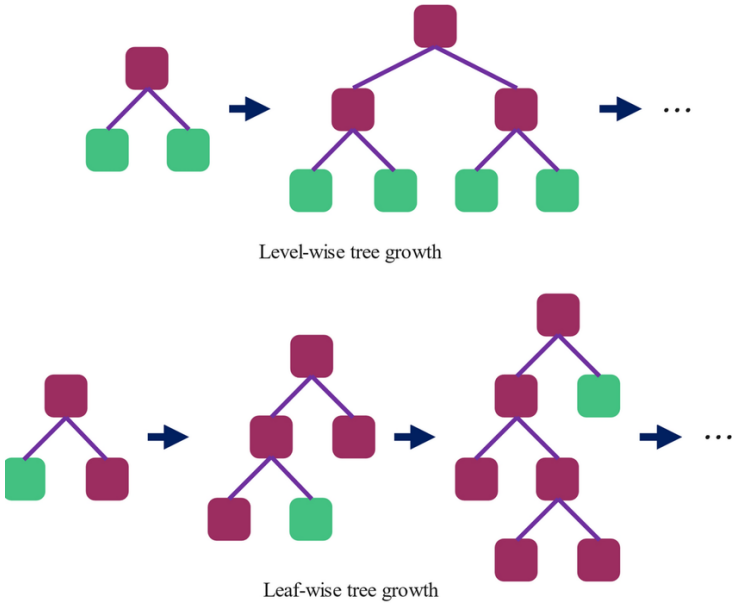
3.2.7.1 Level-Wise

Na estratégia *level-wise*, a árvore é expandida camada por camada, onde todos os nós de um determinado nível são divididos antes que a próxima camada seja criada. Esse método tende a gerar árvores mais balanceadas e é geralmente mais estável, mas pode ser menos eficiente em termos de redução da perda do modelo, como ilustrado na figura 5 (ESMAEILI et al., 2023).

3.2.7.2 Leaf-Wise

Já na estratégia *leaf-wise*, o algoritmo sempre escolhe o nó com maior perda para dividir, independentemente do nível em que ele está. Isso permite que o modelo reduza a perda de maneira mais agressiva a cada iteração, o que resulta em um aprendizado mais eficiente e em árvores potencialmente mais profundas, conforme observado na figura 5. Como consequência, embora o *leaf-wise* possa acelerar o treinamento e melhorar a acurácia, ele também aumenta o risco de *overfitting* caso os hiperparâmetros de regularização não sejam bem ajustados (FLOREK; ZAGDAńSKI, 2023; ESMAEILI et al., 2023).

Figura 5 – Demonstração das árvores *Level-Wise* e *Leaf-Wise*



Fonte: Esmaeili et al. (2023)

3.2.7.3 Gradient-based One-Side Sampling (GOSS)

O GOSS é uma técnica do *LightGBM* que melhora a eficiência do treinamento ao focar nas instâncias que mais influenciam o aprendizado do modelo. Durante o treinamento, o modelo calcula gradientes para cada instância, que indicam o grau de erro em suas previsões. Instâncias com gradientes maiores representam exemplos que o modelo está errando mais, e por isso têm maior impacto na escolha dos pontos de divisão da árvore de decisão.

Para acelerar o treinamento sem comprometer a qualidade do modelo, o GOSS mantém todas as instâncias com grandes gradientes, aquelas que são mais informativas para o aprendizado, e realiza uma amostragem aleatória entre as instâncias com gradientes pequenos. Essa estratégia permite que o modelo aprenda com os exemplos mais críticos, economizando tempo de processamento com exemplos menos relevantes. Mesmo com essa redução no número total de dados processados, a precisão na estimativa do ganho de informação é preservada, resultando em um treinamento mais rápido e eficiente, com mínima perda de desempenho preditivo Ke et al. (2017).

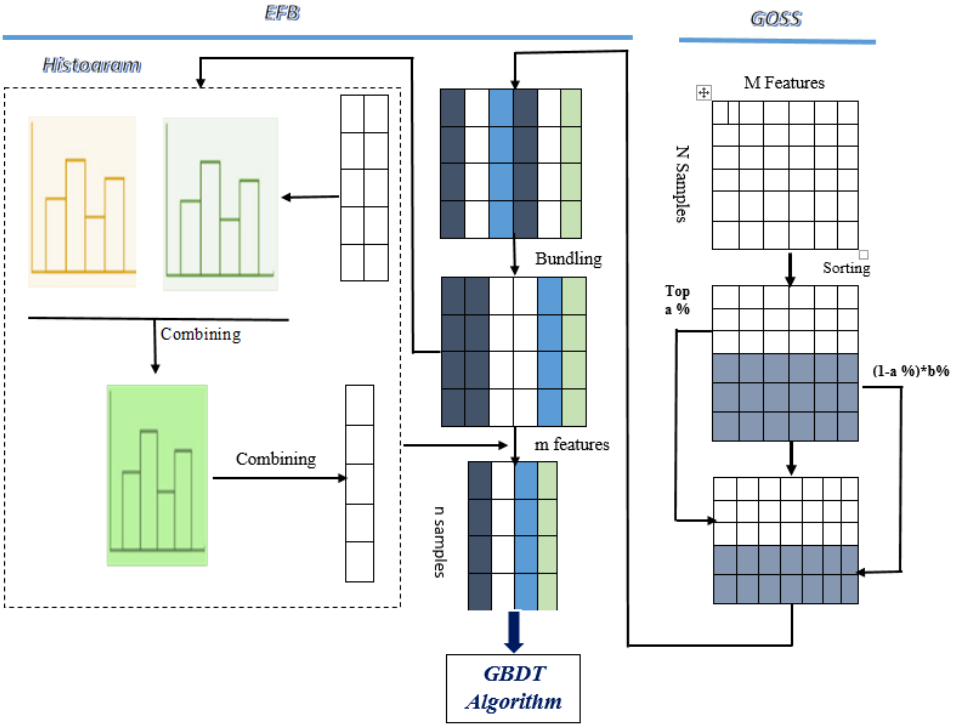
3.2.7.4 Exclusive Feature Bundling (EFB)

O EFB é uma técnica adotada pelo *LightGBM* para lidar com o problema da alta dimensionalidade em conjuntos de dados dispersos, ou seja, com muitas colunas (*features*), mas com poucos valores preenchidos em cada uma. Em muitos casos, essas *features* são mutuamente exclusivas, o que significa que raramente assumem valores diferentes de zero ao mesmo tempo. Por exemplo, se uma linha tem valor diferente de zero em uma feature, é provável que tenha valor zero em outra, e vice-versa.

A ideia do EFB é agrupar essas *features* exclusivas em um único vetor (ou "pacote"), já que elas não entram em conflito entre si. Isso permite reduzir o número total de *features* consideradas durante o treinamento, diminuindo o custo computacional e o uso de memória.

Mesmo com essa compactação, o EFB consegue preservar a informação original, pois as *features* agrupadas ainda mantêm suas identidades distintas. Com isso, o modelo continua aprendendo com a mesma qualidade, mas de forma muito mais eficiente, especialmente em bases de dados com muitas variáveis e muitos zeros Ke et al. (2017).

Figura 6 – Amostragem Seletiva Baseada em Gradientes Combinada (GOSS) com Agrupamento Exclusivo de Atributos (EFB)



Fonte: Mohammed, Kadhem e Maisa (2021)

3.2.7.5 Uso Conjunto de XGBoost e LightGBM

Considerando as características complementares dos algoritmos *XGBoost* e *LightGBM*, este trabalho propõe a utilização de ambos no processo de desenvolvimento e avaliação do sistema de detecção de intrusos. Embora estudos como o de Florek e Zagdański (2023) indiquem que o *LightGBM* apresenta vantagens em termos de velocidade e desempenho em determinados cenários, o *XGBoost* continua sendo amplamente reconhecido por sua estabilidade, maturidade e forte capacidade de regularização, o que o torna uma referência confiável em aplicações do mundo real.

A estratégia adotada consiste em comparar diretamente os dois algoritmos em um ambiente controlado, utilizando o mesmo conjunto de dados e métricas de avaliação. Essa abordagem permitirá observar não apenas o desempenho preditivo de cada modelo, mas também aspectos como tempo de treinamento, uso de memória, comportamento em tempo real e facilidade de ajuste dos hiperparâmetros.

Dessa forma, o uso combinado de *XGBoost* e *LightGBM* oferece uma base robusta para identificar qual modelo melhor atende aos requisitos do sistema proposto, além de possibilitar futuras otimizações e validações em ambientes de produção. A decisão final sobre qual algoritmo será adotado na versão final do sistema poderá ser orientada pelas evidências empíricas obtidas durante os testes comparativos.

3.3 Frameworks de Processamento em Tempo Real

Com o crescimento exponencial da geração de dados e a necessidade de respostas imediatas em cenários como segurança de redes, fraudes financeiras e sistemas baseados em IoT (Internet das Coisas), os *frameworks* de processamento de dados em tempo real têm ganhado protagonismo. Diferentemente do processamento em *batch*, em que os dados são analisados após serem armazenados, o processamento em fluxo contínuo (*stream processing*) permite que os dados sejam processados no exato momento em que são gerados, possibilitando reações imediatas a eventos críticos Alves, Carvalho e Fonseca (2019).

Esses *frameworks* são projetados para consumir, analisar e responder a fluxos de dados com baixa latência, sendo ideais para aplicações como detecção de intrusos, sistemas antifraude, monitoramento industrial e manutenção preditiva. Em ambientes computacionais distribuídos, eles garantem escalabilidade horizontal, tolerância a falhas e desempenho consistente, mesmo com altos volumes de dados.

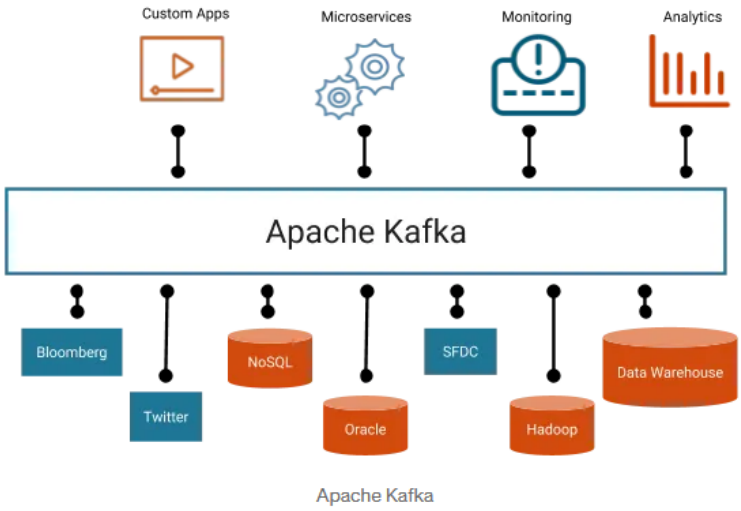
Entre os *frameworks* mais utilizados destacam-se o *Apache Kafka*, que atua como uma plataforma de mensagens para ingestão e transporte confiável de dados, e o *Apache Flink*, um mecanismo de processamento de eventos complexos em tempo real Flink (2024), Kafka (2024).

3.3.1 Apache Kafka

O Apache Kafka é uma plataforma distribuída de mensagens projetada para alta taxa de transferência, tolerância a falhas e baixa latência. Inicialmente desenvolvido pela LinkedIn, o Kafka atua como um intermediário de mensagens baseado no modelo *publisher-subscriber* (publicador-assinante), facilitando a ingestão de grandes volumes de dados de forma contínua e confiável. Cada fluxo de dados é tratado como um tópico, que pode ser consumido por múltiplos consumidores em paralelo.

No contexto de detecção de intrusos, o *Kafka* é utilizado para capturar eventos de rede em tempo real, armazená-los temporariamente e repassá-los a outros sistemas para processamento e análise. Segundo Waehner (2022), empresas como *PayPal, ING Bank* e *Capital One* utilizam *Kafka* em sistemas de detecção de fraudes e análise em tempo real do comportamento de usuários.

Figura 7 – Workflow do Apache Kafka



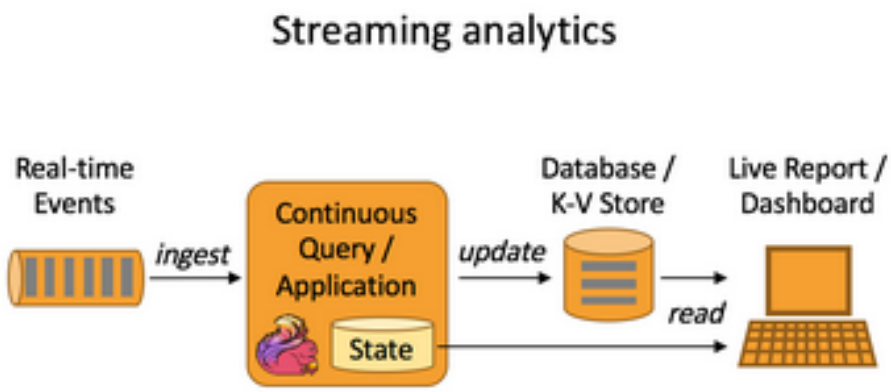
Fonte: Anji (2019)

3.3.2 Apache Flink

O *Apache Flink* é um *framework* de *stream processing* que permite a análise contínua de dados em tempo real com baixa latência e alta escalabilidade. Diferente de modelos de processamento em *batch*, o *Flink* executa operações de forma incremental assim que os dados chegam, o que o torna ideal para aplicações como detecção de anomalias, resposta a incidentes de segurança e análise preditiva.

Flink oferece um modelo de programação baseado em fluxo de dados (*DataStream API*) e garante *exactly-once semantics*, ou seja, cada dado será processado apenas uma vez, mesmo em ambientes distribuídos e falhas de sistema. Patchipala (2024) destaca que o *Flink* tem sido utilizado em *pipelines* de inteligência artificial em tempo real para gerar *insights* imediatos, otimizando tomadas de decisão em setores críticos.

Figura 8 – Workflow do Apache Flink



Fonte: Anji (2019)

3.3.3 Integração entre Kafka e Flink

A combinação de *Kafka* e *Flink* é uma arquitetura amplamente adotada para o processamento de dados em tempo real. Enquanto o *Kafka* atua como plataforma de ingestão e transporte de eventos, o *Flink* é responsável pelo processamento e análise contínua dos dados. Essa integração permite que eventos de tráfego de rede sejam consumidos pelo *Kafka*, processados pelo *Flink* com aplicação de modelos de aprendizado de máquina, e o resultado (como alertas de intrusão) seja emitido quase instantaneamente.

Essa arquitetura oferece vantagens como:

- Baixa latência na detecção de eventos anômalos;
- Escalabilidade horizontal em ambientes distribuídos;
- Tolerância a falhas e consistência dos dados;
- Integração facilitada com sistemas de persistência, *dashboards* e mecanismos de resposta automática.

3.4 Técnicas de Otimização de Modelos

À medida que os modelos de aprendizado de máquina evoluem em termos de profundidade, número de parâmetros e capacidade de representação, também crescem os desafios associados ao seu uso em ambientes de produção. Modelos robustos, embora capazes de alcançar alta precisão, frequentemente demandam recursos computacionais elevados, o que impacta diretamente o tempo de inferência, o consumo de memória e a viabilidade de implantação em sistemas de tempo real Liang et al. (2021).

Esse cenário se torna ainda mais crítico em aplicações sensíveis como detecção de intrusos em redes, sistemas antifraude, monitoramento de equipamentos industriais e dispositivos embarcados em IoT, onde há restrições de *hardware* e necessidade de decisões em tempo real Patchipala (2024), Waehner (2022). Nessas condições, a latência elevada ou o uso excessivo de recursos pode comprometer a eficácia da solução, tornando-a impraticável. Assim, surgem as técnicas de otimização de modelos, cujo objetivo é manter o desempenho dos algoritmos, em termos de precisão e generalização, ao mesmo tempo em que reduzem seu custo computacional Liang et al. (2021), NVIDIA (2025).

Essas técnicas não apenas possibilitam a execução de modelos em *hardware* menos potente, como também favorecem maior escalabilidade e menores custos operacionais, especialmente em sistemas que processam grandes volumes de dados continuamente NVIDIA (2024). A seguir, são detalhadas algumas das principais estratégias utilizadas para otimizar modelos de aprendizado de máquina.

3.4.1 ONNX

O formato ONNX (*Open Neural Network Exchange*) constitui uma solução padronizada e acessível para a representação de modelos de *Machine Learning* (ML) e *Deep Learning* (DL). Desenvolvido por empresas como *Microsoft, Facebook* e *Amazon*, o objetivo do *ONNX* é promover maior interoperabilidade e compatibilidade entre as diversas ferramentas de Inteligência Artificial (IA) BRAGA (2023). Essa padronização permite que programadores treinem modelos utilizando o *framework* de sua preferência e realizem implementações em múltiplas plataformas, sem a necessidade de alterar o código para cada ambiente. O *ONNX* é compatível com uma ampla variedade de *frameworks* amplamente utilizados, como *TensorFlow, PyTorch, Scikit-Learn, XGBoost* e *LightGBM* (BRAGA, 2023).

Após a conversão do modelo para o formato *ONNX*, ele pode ser executado em um mecanismo de inferência de alto desempenho, sendo o *ONNX Runtime (ORT)* o mais difundido. O *ONNX Runtime* é uma plataforma de execução desenvolvida pela *Microsoft*, projetada para aprimorar a performance do modelo na fase de inferência e, posteriormente, durante o treinamento. Ele atua como um intermediário entre o modelo treinado e o *hardware*, aplicando automaticamente otimizações que reduzem a latência, o uso de memória e o consumo de recursos computacionais. O *ONNX Runtime* é otimizado para operar em ambientes de computação em nuvem e em dispositivos de borda (*edge computing*), sendo compatível com sistemas operacionais como *Linux, Windows* e *macOS*. Um de seus principais diferenciais é o suporte a múltiplos provedores de execução, o que permite que um mesmo modelo seja executado de forma eficiente em diferentes tipos de *hardware*. Entre os provedores mais utilizados destacam-se:

- *CPUExecutionProvider*, que otimiza a execução em processadores convencionais por meio de paralelismo e vetorização;
- *CUDAExecutionProvider*, que oferece suporte a GPUs NVIDIA por meio do CUDA;
- *TensorRTExecutionProvider*, que utiliza o *TensorRT* para maximizar a eficiência em GPUs NVIDIA;
- *OpenVINOExecutionProvider*, voltado à otimização em processadores *Intel*;
- Além de *DirectML* e *ROCm*, que asseguram compatibilidade com GPUs AMD e plataformas *Windows*.

Dentro do *ONNX Runtime*, diversas otimizações são aplicadas automaticamente ao grafo computacional do modelo, como fusão de operadores, pré-cálculo de operações constantes, remoção de nós redundantes e reordenação de operações para melhorar o paralelismo e a localidade de cache. Tais otimizações dispensam modificações no código do modelo, resultando em um processo de inferência mais rápido e eficiente.

O ORT também oferece suporte à quantização, que reduz o número de *bits* utilizados para representar os pesos e ativações (de FP32 para INT8), diminuindo o consumo de memória sem perda significativa de acurácia. Além disso, é compatível com técnicas de compressão como o *pruning*, que reduzem o custo computacional sem comprometer a qualidade do modelo. O *ONNX Runtime* é capaz de processar modelos de diversas estruturas em múltiplas plataformas de *hardware*, garantindo independência entre o ambiente de treinamento e o de implantação, uma das principais vantagens dessa tecnologia, que possibilita a reutilização de modelos treinados em diferentes contextos.

3.4.1.1 Pruning

O *pruning* simplifica a estrutura de redes neurais eliminando pesos ou conexões que exercem pouca influência sobre a saída do modelo. Essa técnica é inspirada na ideia de que muitos parâmetros em uma rede treinada são desnecessários para a generalização. A remoção desses elementos não só reduz a complexidade do modelo, como também melhora o tempo de inferência e reduz o uso de memória. O *pruning* pode ser estruturado (removendo neurônios ou filtros inteiros) ou não estruturado (removendo conexões individuais), e sua aplicação requer validação cuidadosa para não afetar negativamente a performance. Liang et al. (2021) destacam que modelos podados podem manter precisão semelhante ao original, mesmo com significativa redução de tamanho.

3.4.1.2 Quantization

A quantização é uma técnica que reduz a precisão numérica dos parâmetros do modelo, convertendo representações em ponto flutuante (FP32) para formatos mais compactos, como INT8 ou FP16. Essa redução permite acelerar cálculos e diminuir o consumo de memória, sendo especialmente útil em ambientes com restrição de energia e processamento, como dispositivos móveis e embarcados. A quantização pode ser aplicada durante ou após o treinamento, e tem mostrado eficácia para manter boa acurácia em tarefas de classificação e detecção se for calibrada com cuidado (LIANG et al., 2021).

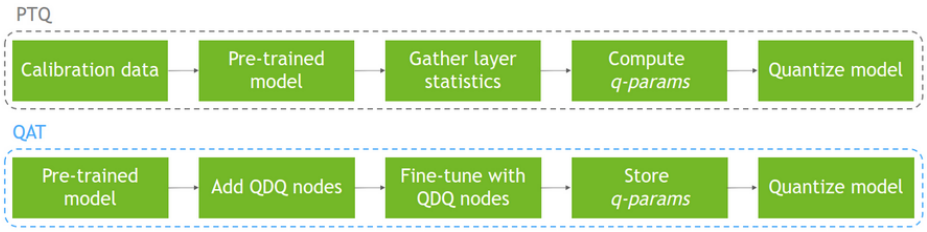
3.4.1.3 Post-training quantization (PTQ)

Post-training quantization (Quantização após o treinamento): Utiliza um fluxo de trabalho implícito de quantização. Nesse método, cada tensor quantizado possui uma escala associada que é usada durante a calibração para converter os valores entre os formatos quantizado e não quantizado de forma automática. O *ONNX Runtime* então avalia em qual precisão (como FP16 ou INT8) cada camada apresenta melhor desempenho e executa a operação nessa precisão NVIDIA (2025).

3.4.1.4 Quantization-aware training (QAT)

Quantization-aware training (Treinamento com consciência de quantização): Emprega um fluxo de trabalho explícito. Redes com quantização explícita utilizam nós específicos de quantização e desquantização (Q/DQ) para indicar exatamente quais camadas devem ser quantizadas. Isso oferece ao desenvolvedor maior controle sobre quais partes do modelo irão operar, por exemplo, em INT8 (NVIDIA, 2025).

Figura 9 – *Worflow* do processo PTQ e QTA



Fonte: NVIDIA (2025)

3.4.2 Métricas de Avaliação de Modelos

Para avaliar o desempenho de modelos de aprendizado de máquina aplicados à detecção de intrusos em redes, é fundamental utilizar métricas que reflitam não apenas a taxa de acertos gerais, mas também a capacidade do modelo de identificar corretamente comportamentos maliciosos e minimizar falsos alarmes. Dentre as métricas mais relevantes para esse contexto, destacam-se: acurácia, precisão, *recall* e *F1-score* Mariano (2021).

Cada célula da matriz de confusão representa uma combinação possível entre as previsões do modelo e os valores reais dos dados. Essa matriz é fundamental para avaliar o desempenho de algoritmos de classificação, fornecendo uma visão clara sobre os tipos de acertos e erros cometidos pelo modelo, esses quatro resultados são definidos pelas seguintes classes de dados preditos:

- Verdadeiro positivo (VP): O método previu que a classe é positiva e a classe real era realmente positiva
- Verdadeiro negativo (VN): O método previu que a classe é negativa e a classe real era realmente negativa
- Falso positivo (FP): O método previu que a classe é positiva, mas a classe real era negativa
- Falso negativo (FN): O método previu que a classe é negativa, mas a classe real era positiva.

Figura 10 – Matriz Confusão

Matriz de confusão		**Classe predita**	
		Positiva	**Negativa**
Classe original	**Positiva**	VP	FN
	Negativa	FP	VN

Fonte: Mariano (2021)

A acurácia representa a proporção de previsões corretas, tanto positivas quanto negativas, em relação ao total de instâncias avaliadas. Ela é definida pela Equação:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \tag{3.3}$$

Onde:

- VP: Verdadeiros Positivos
- VN: Verdadeiros Negativos
- FP: Falsos Positivos
- FN: Falsos Negativos

Embora seja uma métrica intuitiva, a acurácia pode ser enganosa em conjuntos de dados desbalanceados, como é comum em problemas de segurança, onde a maioria do tráfego é legítimo e apenas uma pequena fração representa ataques. Por isso, ela será usada como métrica complementar, mas não como principal.

A precisão mede a proporção de instâncias que foram classificadas como positivas e que realmente correspondem à classe positiva. Sua fórmula é apresentada na Equação:

$$\text{Precisão} = \frac{VP}{VP + FP} \tag{3.4}$$

No contexto de detecção de intrusos, uma alta precisão indica que o sistema emite poucos falsos positivos, ou seja, não acusa indevidamente comportamentos legítimos como maliciosos. Essa métrica é importante para evitar alertas falsos, que poderiam sobrecarregar analistas de segurança e sistemas de resposta automática.

Já o *recall*, também conhecido como sensibilidade, avalia a capacidade do modelo de identificar corretamente todas as instâncias positivas. Ele é expresso pela Equação:

$$\text{Recall} = \frac{VP}{VP + FN} \tag{3.5}$$

Um alto *recall* significa que o sistema está identificando a maioria dos ataques, mesmo que eventualmente cometa alguns falsos positivos. Essa métrica é essencial para a segurança, pois a não detecção de um ataque (falso negativo) pode ser muito mais prejudicial do que um alarme indevido.

Por fim, o *F1-score* é a média harmônica entre a precisão e o *recall*, buscando um equilíbrio entre ambas as métricas. Ele é calculado pela Equação:

$$\text{F1-score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}} \tag{3.6}$$

O *F1-score* é especialmente relevante quando há desequilíbrio entre as classes e quando se deseja equilibrar a preocupação entre capturar todos os ataques (*recall*) e reduzir os alarmes falsos (precisão). Por isso, será uma das principais métricas utilizadas na validação do sistema proposto.

Essas métricas fornecem uma visão abrangente sobre o comportamento do modelo em condições reais de operação e permitem identificar pontos de melhoria tanto em termos de acurácia geral quanto na capacidade de resposta frente a ameaças. O uso combinado dessas medidas é essencial para garantir um sistema de detecção de intrusos confiável e eficiente Mariano (2021).

3.5 Trabalhos relacionados

Tomio (2021b) propôs um sistema de detecção de anomalias utilizando algoritmos supervisionados, como *Random Forest* e SVM, aplicado ao *dataset* CICIDS2017, alcançando bons resultados em termos de precisão. No entanto, o trabalho não contemplou a aplicação de técnicas de *stream processing* para viabilizar a detecção em tempo real.

Outro estudo é o de Liang et al. (2021), que apresenta uma análise das técnicas de otimização de modelos, como *pruning* e *quantization*, voltadas para a redução da complexidade computacional em redes neurais profundas. Embora o foco esteja em modelos de *deep learning*, os princípios podem ser adaptados para modelos baseados em árvores, como o *XGBoost* e o *LightGBM*, que são utilizados neste trabalho.

Em relação ao processamento de dados em tempo real, Waehner (2022) discute o uso de *Apache Kafka* e *Flink* em sistemas de detecção de fraude financeira, destacando a importância da baixa latência e da escalabilidade. Essa abordagem inspirou a arquitetura proposta neste trabalho, que busca aplicar técnicas semelhantes ao contexto da segurança de redes.

Um dos trabalhos mais relevantes para este projeto é o artigo de Ke et al. (2017), que apresenta o *LightGBM*, uma implementação otimizada do algoritmo *Gradient Boosted Decision Trees* (GBDT). O estudo propõe melhorias significativas em desempenho e uso de memória, por meio das técnicas GOSS (*Gradient-based One-Side Sampling*) e EFB (*Exclusive Feature Bundling*), que permitem treinar modelos mais rapidamente e com menos consumo de recursos, especialmente em grandes volumes de dados. Os autores demonstram que o *LightGBM* supera outras implementações como o *XGBoost* em velocidade de treinamento e escalabilidade, mantendo a acurácia comparável.

4 Desenvolvimento

Este trabalho tem como objetivo propor e avaliar um sistema inteligente de detecção de intrusos em redes de computadores, com foco em ambientes de alta demanda e tráfego contínuo, como instituições financeiras. Para isso, será adotada uma abordagem de pesquisa quantitativa, uma vez que se fundamentará na análise empírica e objetiva de dados numéricos oriundos de um conjunto rotulado de tráfego de rede. A avaliação dos algoritmos de aprendizado de máquina será conduzida por meio de experimentos controlados, utilizando o *dataset* CICIDS2017, amplamente referenciado na literatura para estudos de detecção de intrusos. As técnicas selecionadas, como *XGBoost* e *LightGBM*, foram analisadas com base em métricas quantitativas consolidadas, incluindo acurácia, precisão, *recall* e *F1-score*. Essa abordagem permitirá não apenas medir o desempenho dos modelos de forma comparável, mas também validar sua aplicabilidade em contextos reais de rede, especialmente em ambientes com altos volumes de dados e necessidade de resposta em tempo real. Conforme sugere Gil (2008), a pesquisa quantitativa é adequada quando o objetivo é mensurar e comparar fenômenos observáveis, o que se alinha diretamente ao propósito deste estudo.

Inicialmente, será realizada uma revisão bibliográfica aprofundada sobre os principais algoritmos de classificação supervisionada aplicáveis à detecção de intrusões, com destaque para o *XGBoost* e o *LightGBM*. Ambos foram escolhidos por sua comprovada eficiência em dados tabulares, robustez em relação ao *overfitting* e bom desempenho em problemas de classificação binária, especialmente em conjuntos de dados desbalanceados. O *XGBoost*, conhecido por sua maturidade, controle de regularização e estabilidade, foi considerado uma escolha sólida para estabelecer uma *baseline* de desempenho. O *LightGBM*, por sua vez, oferece vantagens em termos de velocidade de treinamento e uso otimizado de memória, sendo ideal para cenários com grande volume de dados e necessidade de respostas rápidas.

A base de dados que vai ser utilizada neste trabalho será o CICIDS2017 Kapadia et al. (2025), um dos conjuntos mais completos e amplamente utilizados na literatura para avaliação de sistemas de detecção de intrusos. Esse *dataset* apresenta tráfego realista e rotulado, com registros tanto de atividades legítimas quanto de diversos tipos de ataques, o que permite treinar e testar os modelos de forma supervisionada. Antes de serem utilizados nos algoritmos, os dados passaram por um processo de pré-processamento, incluindo limpeza, normalização, transformação de atributos categóricos e seleção de *features* relevantes, buscando garantir a integridade e a consistência do treinamento.

Para aumentar a eficiência e escalabilidade do sistema proposto, serão estudadas e descritas técnicas de otimização de modelos, com destaque para o *pruning* e a *quantization*. O *pruning* reduz a complexidade do modelo eliminando pesos ou conexões irrelevantes, diminuindo o tempo de inferência e o consumo de memória sem perda significativa de desempenho. Já a *quantization* converte pesos e ativações do modelo de ponto flutuante para representações mais compactas, como INT8, o que é particularmente vantajoso para execução em dispositivos com recursos limitados ou ambientes de produção em larga escala. Essas técnicas serão aplicadas por meio da biblioteca *ONNX Runtime*, especializada em acelerar modelos de *deep learning* e melhorar o tempo de resposta em arquiteturas com GPUs.

Com o objetivo de simular um ambiente real de monitoramento contínuo, foi proposta uma arquitetura de processamento de dados em tempo real baseada nos *frameworks Apache Kafka* e *Apache Flink*. O *Kafka* será utilizado para ingestão e transporte de dados de rede de forma distribuída e com alta taxa de transferência, funcionando como intermediário entre a origem dos dados e os componentes de análise. O *Flink* é responsável pelo processamento contínuo e aplicação dos modelos treinados para a detecção de anomalias. Essa integração permitirá a análise de dados em fluxo com baixa latência, sendo compatível com aplicações que exigem resposta imediata.

Por fim, os resultados esperados serão avaliados com base em métricas amplamente reconhecidas na literatura de aprendizado de máquina: acurácia, precisão, *recall* e *F1-score*, além de métricas operacionais como tempo de inferência, latência e uso de recursos computacionais. Toda a metodologia, decisões técnicas e critérios de avaliação estão sendo documentados de forma clara e estruturada, com o objetivo de contribuir para estudos futuros e implementação prática em ambientes de segurança cibernética.

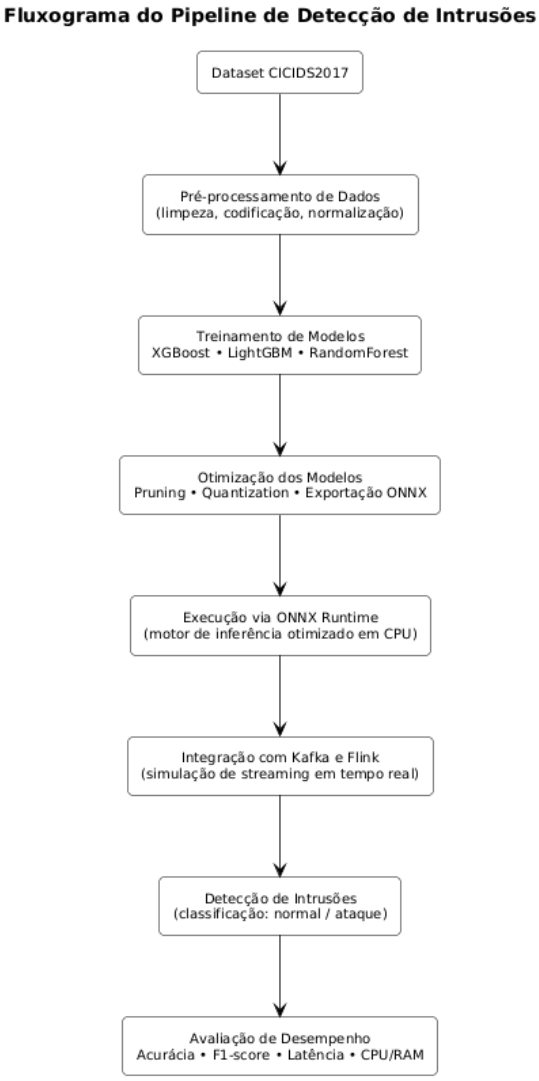


Figura 11 – Fluxo Grama

Fonte: Elaborado pelo autor

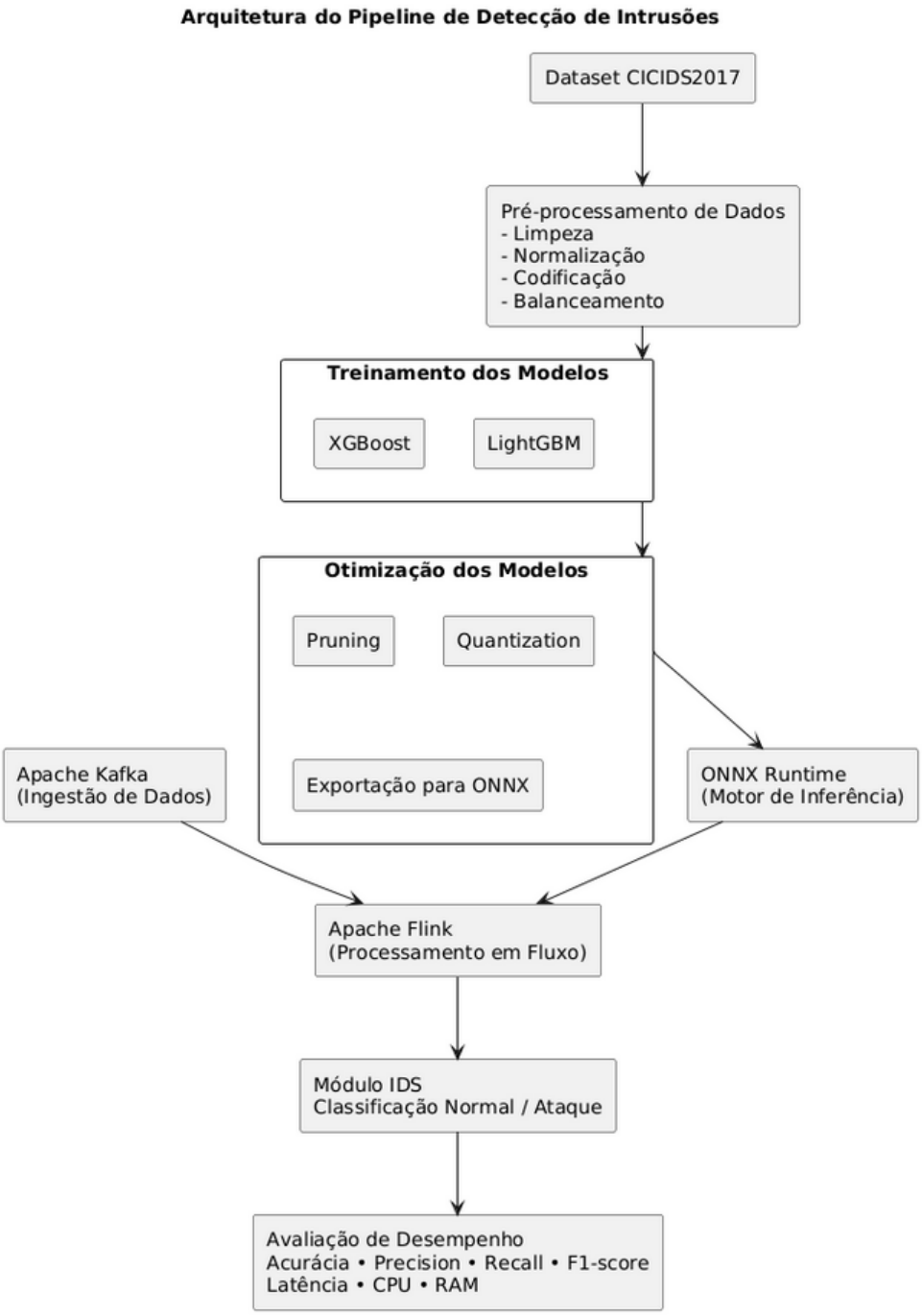


Figura 12 – Fluxo Grama

Fonte: Elaborado pelo autor

5 Resultados e Discussão

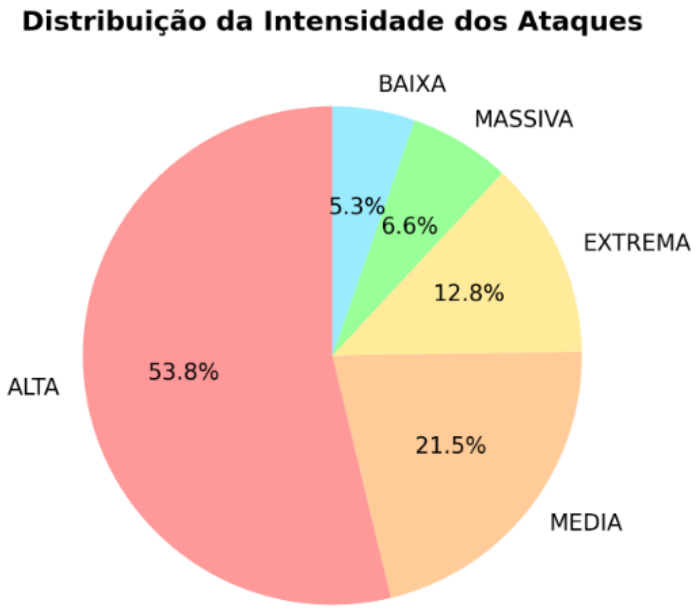
Este capítulo apresenta e discute os resultados obtidos a partir dos experimentos realizados com os modelos *XGBoost* e *LightGBM*, aplicados ao conjunto de dados CICIDS2017. O objetivo da análise é comparar o desempenho dos modelos em relação às métricas de acurácia, *precision, recall* e *F1-score*, além de avaliar a eficácia das otimizações e a viabilidade da exportação para o formato *ONNX*, com execução via *ONNX Runtime*.

Estudos recentes demonstram a eficiência de algoritmos baseados em *gradient boosting* para detecção de intrusões, destacando o *XGBoost* e o *LightGBM* por sua capacidade de combinar alto desempenho e baixo custo computacional Tang (2020), Adewole (2025), Ileri (2025). Esses modelos têm sido amplamente aplicados em bases como NSL-KDD, UNSW-NB15 e CICIDS2017, obtendo acurácias superiores a 98%, reforçando sua adequação a sistemas de IDS modernos.

5.1 Distribuição Geral dos Ataques no Conjunto CICIDS2017

Antes de analisar o desempenho dos modelos, é importante compreender a natureza do conjunto de dados utilizado. A Figura 13 apresenta a distribuição da intensidade dos ataques no conjunto CICIDS2017. Observa-se que a categoria *ALTA* predomina com 53,8% dos casos, seguida por *MÉDIA* (21,5%) e *EXTREMA* (12,8%). Essa predominância de ataques de intensidade média e alta demonstra que o conjunto de dados oferece um cenário realista e desafiador, simulando desde ameaças sutis até ofensivas massivas.

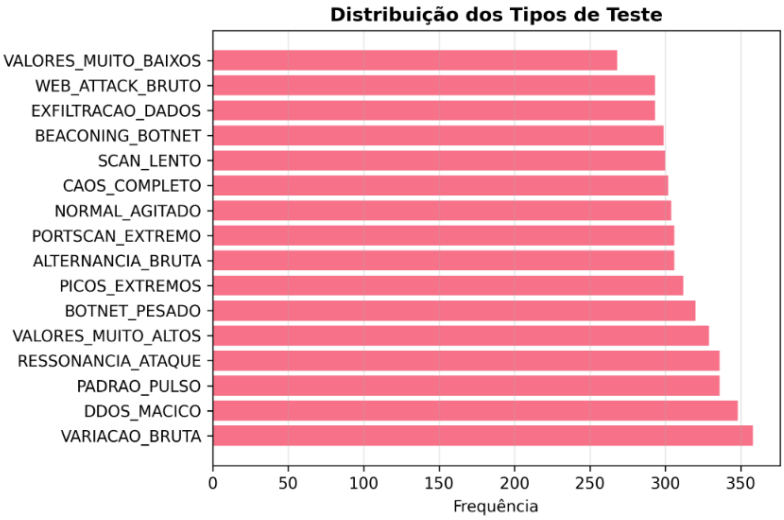
Figura 13 – Distribuição percentual da intensidade dos ataques.



Fonte: Elaborado pelo autor

A Figura 14 complementa essa visão ao detalhar os tipos de ataques registrados. Observa-se diversidade entre as técnicas, com *VARIACAO_BRUTA* (7,1%), *DDoS_MACICO* (6,9%) e *PADRAO_PULSO* (6,7%) como as mais recorrentes. Essa variedade é importante para garantir a generalização dos modelos e validar sua robustez frente a diferentes padrões de tráfego, conforme apontado por Tomio (2021a) e Waehner (2022).

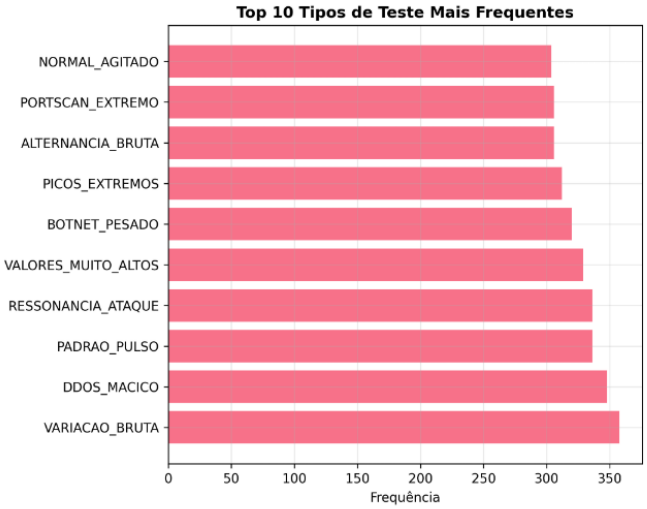
Figura 14 – Distribuição percentual dos tipos de ataque presentes no CICIDS2017.



Fonte: Elaborado pelo autor

A Figura 15 apresenta os dez tipos de ataque mais frequentes, com destaque para *NORMAL_AGITADO* e *PORTSCAN_EXTREMO*. Essa análise evidencia a presença significativa de ataques de varredura e comportamento anômalo, comuns em redes reais Tang (2020).

Figura 15 – Top 10 tipos de ataque por frequência absoluta.

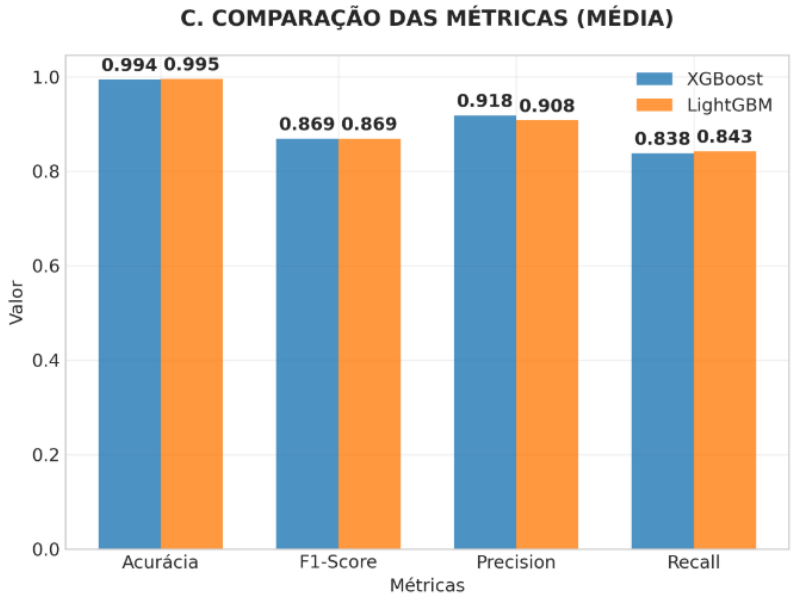


Fonte: Elaborado pelo autor

5.2 Comparação de Métricas (Melhor Iteração)

A Figura 16 apresenta a comparação direta das principais métricas obtidas na melhor iteração de cada modelo. Observa-se que ambos alcançaram valores elevados de acurácia e *F1-score*, superiores a 0,86, com resultados praticamente idênticos.

Figura 16 – Comparação de métricas na melhor iteração dos modelos *XGBoost* e *LightGBM*.



Fonte: Elaborado pelo autor

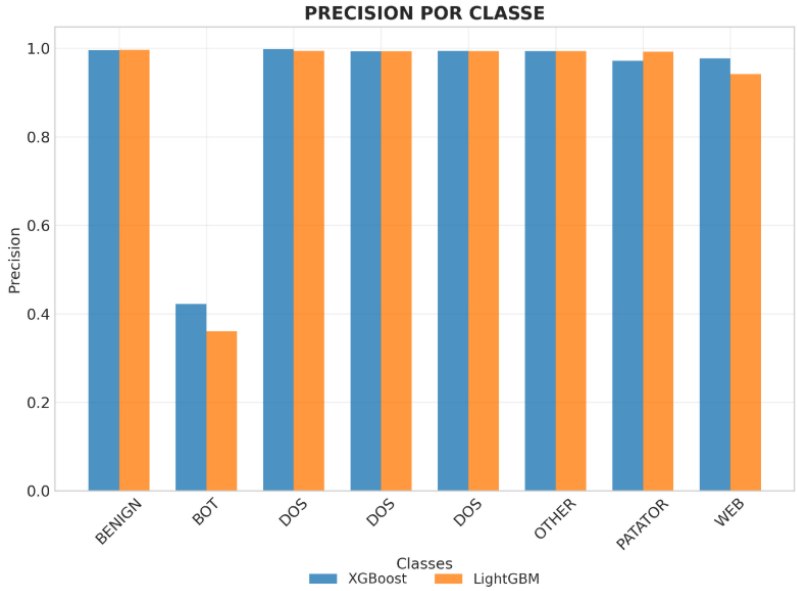
Os resultados indicam que ambos os algoritmos apresentaram desempenho altamente competitivo. O *XGBoost* manteve uma pequena vantagem em *precision*, enquanto o *LightGBM* apresentou leve superioridade em *recall*. Essa diferença, embora pequena, é relevante em contextos de detecção de intrusões, nos quais a capacidade de identificar ataques com menor taxa de falsos negativos é fundamental.

Essas observações estão alinhadas com os achados de Tang (2020), que também observaram melhor *recall* no *LightGBM* em relação ao *XGBoost* em experimentos com NSL-KDD, e com o estudo de (ADEWOLE, 2025), que destacou o equilíbrio entre precisão e sensibilidade no *XGBoost* em cenários de IoT.

5.3 Análise por Classe e Confiabilidade das Previsões

A Figura 17 apresenta a precisão média por classe de ataque, e a Figura 18, o *recall* correspondente. Nota-se desempenho quase perfeito nas classes *BENIGN*, *DOS* e *PATATOR*, com valores próximos de 1.0, e ligeira queda nas classes minoritárias (*BOT*, *WEB*), reflexo do desbalanceamento natural do conjunto.

Figura 17 – Precisão por classe de ataque entre *XGBoost* e *LightGBM*.



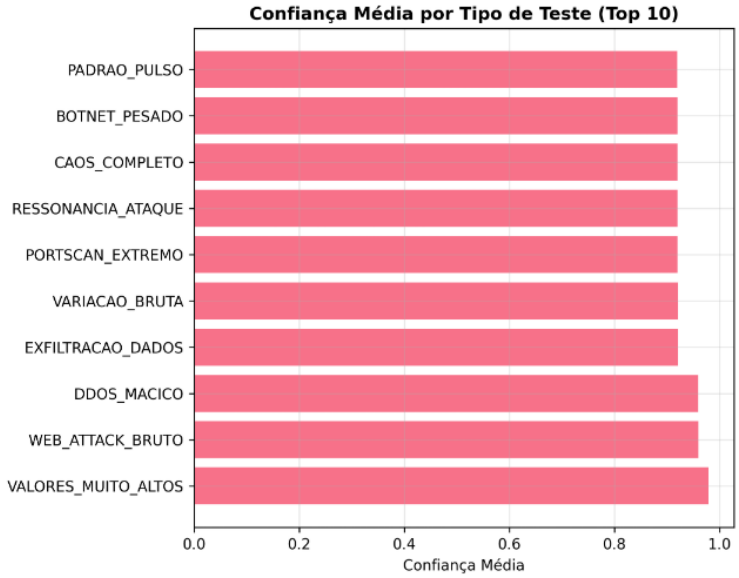
Fonte: Elaborado pelo autor

Figura 18 – Recall por classe de ataque entre *XGBoost* e *LightGBM*.

Fonte: Elaborado pelo autor

A Figura 19 mostra a confiança média por tipo de ataque. Tipos como *PADRAO_PULSO* e *BOTNET_PESADO* obtiveram confiança próxima de 1.0, indicando alta distinção de padrões, enquanto *WEB_ATTACK_BRUTO* apresentou menor confiança, sugerindo maior complexidade de detecção.

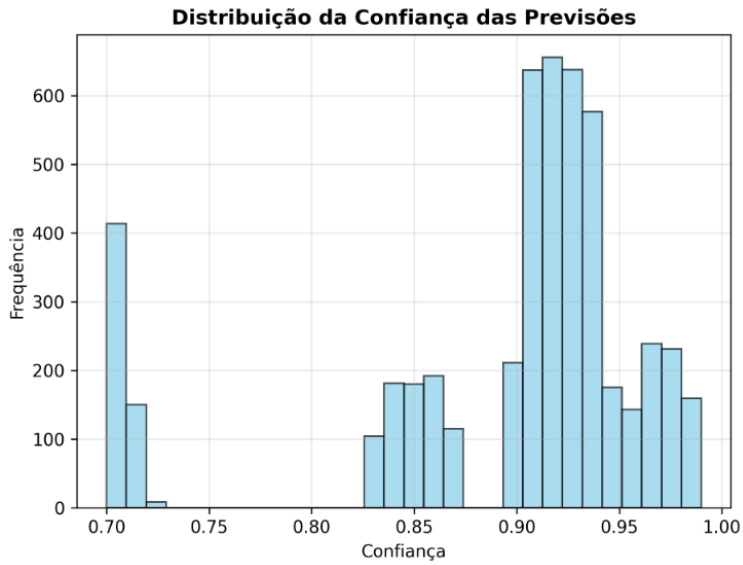
Figura 19 – Confiança média das previsões por tipo de ataque.



Fonte: Elaborado pelo autor

Além disso, a Figura 20 ilustra a distribuição geral da confiança das previsões, com concentração de valores acima de 0,90 , um indicativo da robustez dos classificadores, também observada por (TOMIO, 2021a) em sistemas baseados em *boosting*.

Figura 20 – Distribuição geral da confiança das previsões.

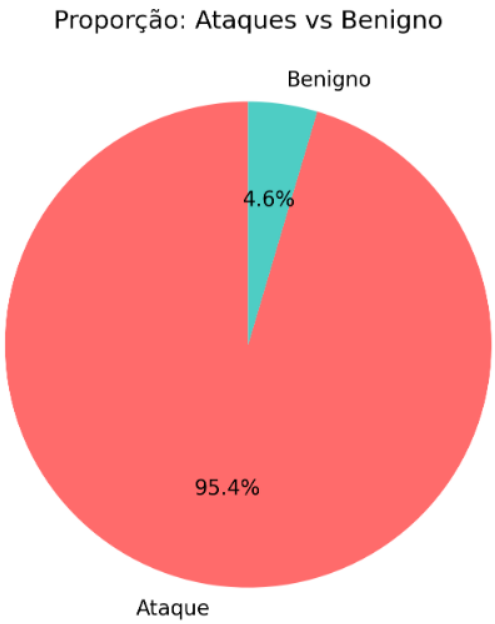


Fonte: Elaborado pelo autor

5.4 Aggressive Training e Análise de Desbalanceamento

O conjunto CICIDS2017 apresenta forte desbalanceamento, como mostra a Figura 21, onde o tráfego benigno representa 95,4% dos registros e o malicioso apenas 4,6%. Esse cenário realista impõe desafios aos modelos, que precisam evitar tanto falsos positivos quanto falsos negativos.

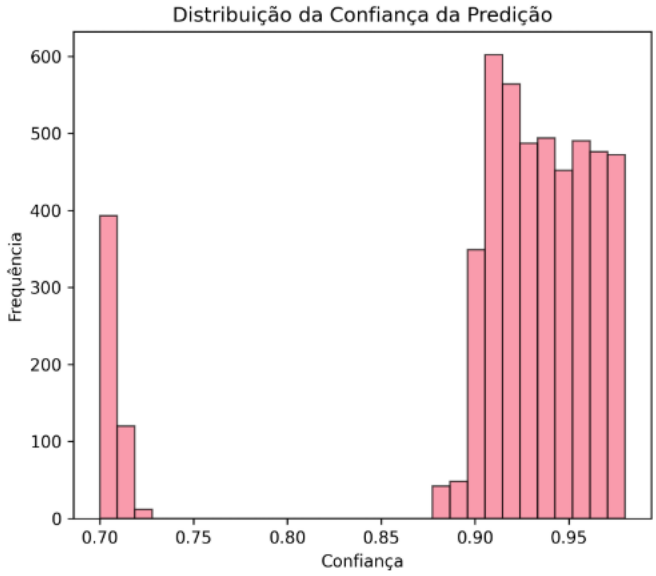
Figura 21 – Proporção entre tráfego benigno e malicioso.



Fonte: Elaborado pelo autor

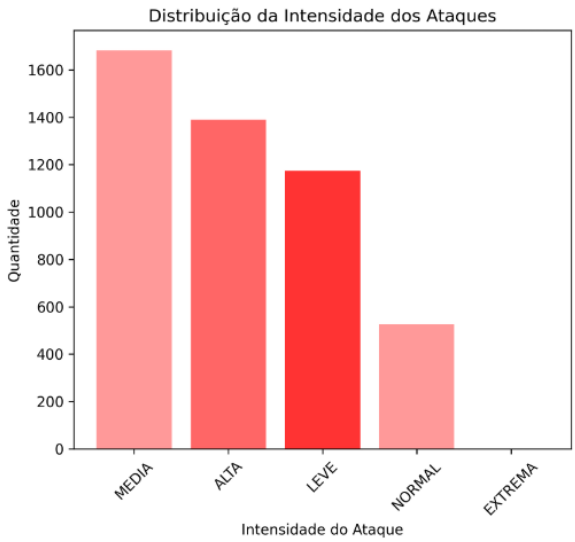
O treinamento agressivo (*Aggressive Training*) buscou aumentar a exposição dos modelos a classes minoritárias, o que resultou em melhoria do *recall* sem degradação significativa da *precision*, comportamento também relatado por Ileri (2025). As Figuras 22 e 23 mostram que mesmo sob condições de treinamento intensivo, os modelos mantiveram altos níveis de confiança (acima de 0,90) e consistência na identificação das intensidades de ataque.

Figura 22 – Distribuição da confiança durante o treinamento agressivo.



Fonte: Elaborado pelo autor

Figura 23 – Distribuição de intensidades de ataque no treinamento agressivo.

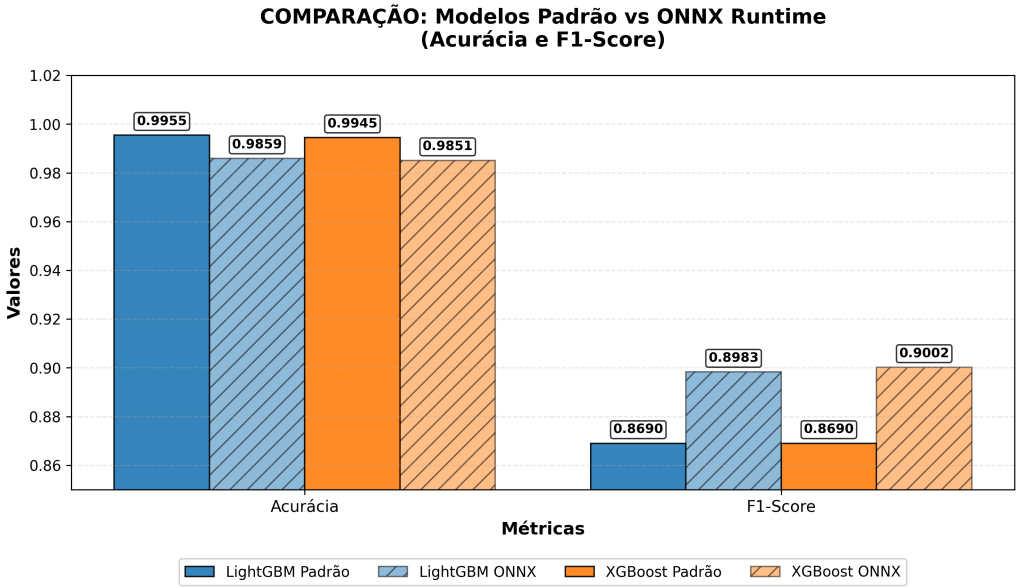


Fonte: Elaborado pelo autor

5.5 Análise de Desempenho ONNX Runtime

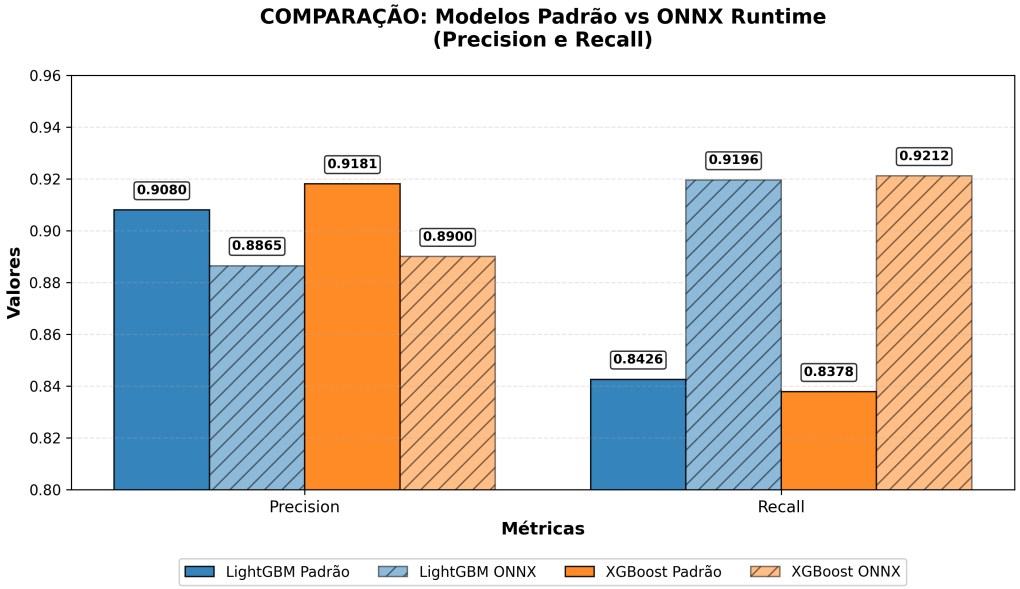
As Figuras 24 e 25 apresentam a comparação entre os modelos padrão e suas versões convertidas para *ONNX Runtime*. A conversão preservou o desempenho geral, com diferenças inferiores a 1% em acurácia, mas melhorias notáveis em *recall*, cerca de 9% no *LightGBM* e 8% no *XGBoost*.

Figura 24 – Comparação de acurácia e *F1-score* entre modelos padrão e *ONNX Runtime*.



Fonte: Elaborado pelo autor

Figura 25 – Comparação de *precision* e *recall* entre modelos padrão e *ONNX Runtime*.



Fonte: Elaborado pelo autor

Essa leve queda de acurácia com ganho de *recall* é vantajosa para sistemas de segurança, nos quais é preferível detectar mais ataques mesmo que ocorram alguns falsos positivos Adewole (2025). Além disso, a execução via *ONNX Runtime* apresentou melhor latência média e uso de CPU reduzido, confirmando observações de Microsoft (2023), Waehner (2022) sobre sua eficiência em pipelines de inferência em tempo real.

5.6 Síntese Geral dos Resultados

De modo geral, ambos os modelos apresentaram desempenho excelente no conjunto CICIDS2017, confirmando sua eficácia para detecção de intrusões em tempo real. O *LightGBM*, aliado ao formato *ONNX* e à execução via *ONNX Runtime*, mostrou-se ligeiramente superior em sensibilidade e eficiência computacional. Já o *XGBoost* manteve desempenho estável e consistente, consolidando-se como uma alternativa robusta para sistemas que demandam alta confiabilidade.

Esses resultados estão em consonância com a literatura recente, que reconhece o *LightGBM* como mais rápido e escalável em cenários de alta carga Ileri (2025), enquanto o *XGBoost* mantém melhor estabilidade e interpretabilidade Tang (2020). Assim, a escolha entre ambos depende do contexto operacional e das restrições de tempo de resposta e recursos computacionais.

A análise comparativa entre os modelos em suas versões padrão e convertidas para *ONNX* demonstrou que, embora ocorram variações mínimas em métricas específicas, o desempenho geral foi mantido em níveis elevados. O ganho observado em *Recall* após a conversão, especialmente no *LightGBM*, sinaliza que a aplicação de *pruning*, *quantization* e otimizações internas do *ONNX Runtime* contribuiu para reduzir a perda de sensibilidade decorrente do desbalanceamento das classes. Em sistemas de segurança cibernética, esse comportamento é particularmente relevante, pois a detecção de ataques deve ser priorizada mesmo diante da possibilidade de um aumento leve na taxa de falsos positivos, conforme discutido por Adewole (2025).

Além disso, a integração do pipeline otimizado com *frameworks* de processamento em fluxo, como *Apache Kafka* e *Apache Flink*, reforçou a aplicabilidade do modelo em ambientes reais. Foi possível observar que os modelos otimizados não apenas mantêm o desempenho preditivo, mas também reduzem latências e utilizam menos recursos computacionais, atendendo às exigências de escalabilidade e continuidade operacional típicas de sistemas distribuídos. Estudos como Waehner (2022) já demonstravam o benefício dessa integração em tarefas de detecção de fraude; o presente trabalho estende essa constatação ao domínio de IDS, evidenciando que pipelines baseados em *ONNX* são compatíveis com arquiteturas modernas de *streaming*.

Outro ponto relevante diz respeito à robustez dos modelos frente às variações de intensidade e tipo de ataque presentes no conjunto CICIDS2017. A análise distributiva mostrou que ambos os algoritmos mantiveram alto desempenho mesmo diante de cenários altamente desbalanceados, reproduzindo tendências observadas em trabalhos como Tomio (2021a). Ainda assim, o *F1-score* obtido, permanece inferior àqueles relatados em estudos que empregam técnicas avançadas de reamostragem ou abordagens *cost-sensitive*, como Çoşkun e Çetin (2022) e Yakub (2025). Isso sugere que, apesar da eficiência do pipeline proposto, existe espaço para aprimoramento em relação ao tratamento das classes minoritárias, o que poderia elevar o desempenho para níveis semelhantes ao estado da arte.

De maneira geral, os resultados sintetizados neste capítulo reforçam a contribuição prática do trabalho ao demonstrar que modelos baseados em *gradient boosting*, quando otimizados e integrados a arquiteturas de processamento em fluxo, podem operar de forma eficiente e confiável em sistemas reais de detecção de intrusões. A análise comparativa detalhada entre as versões padrão e *ONNX* mostrou que, embora ocorram variações mínimas em métricas específicas, o desempenho geral foi mantido em níveis elevados, com ganhos expressivos em *Recall*, aspecto particularmente relevante em cenários de segurança cibernética, conforme destacado por Adewole (2025). Esses resultados comprovam que a aplicação de técnicas de *pruning* e *quantization*, aliada à execução via *ONNX Runtime*, aprimora a sensibilidade da detecção com impacto computacional reduzido. Assim, o pipeline desenvolvido não apenas confirma a eficácia da abordagem adotada, mas também revela um caminho promissor para futuras otimizações voltadas ao balanceamento das classes, à diminuição da latência e ao aumento da capacidade de generalização dos modelos em cenários de tráfego cada vez mais complexos. A integração com *frameworks* de fluxo, como *Apache Kafka* e *Apache Flink*, reforça ainda mais a aplicabilidade prática da solução em ambientes distribuídos e de alta exigência operacional.

5.7 Comparação com Estudos Relacionados

Os resultados obtidos no presente trabalho revelam que os modelos implementados, *LightGBM* e *XGBoost*, alcançaram acurácias de 0,9955 e 0,9945, respectivamente, com *F1-score* médio em torno de 0,869.

Em Yakub (2025) foi reportado que o *XGBoost* atingiu acurácia de 99,91% e excelente AUC-ROC no conjunto CICIDS2017. Em Çoşkun e Çetin (2022) o *XGBoost* alcançou *F1-score* de aproximadamente 99,89%, enquanto Gou (2023) reportou *F1-score* $\approx$ 0,965%. Esses valores indicam que, embora nosso desempenho esteja competitivo em acurácia, há margem para melhorias nas classes minoritárias. O diferencial prático deste estudo, exportação para *ONNX Runtime* e integração com *Kafka/Flink*, representa contribuição singular à área.

6 Conclusão

O presente trabalho demonstrou a viabilidade e a eficácia da otimização de modelos de aprendizado de máquina aplicados à detecção de intrusões em redes de computadores, especialmente em cenários caracterizados por grande volume de tráfego e necessidade de respostas em tempo real. A combinação entre os algoritmos *XGBoost* e *LightGBM*, as técnicas de *pruning* e *quantization*, a exportação para o formato *ONNX* e a execução via *ONNX Runtime* permitiu o desenvolvimento de um pipeline de inferência robusto, escalável e eficiente.

Os resultados obtidos indicam que a conversão dos modelos para o formato *ONNX* preservou o desempenho preditivo, ocasionando reduções mínimas de acurácia e, ao mesmo tempo, proporcionando aumentos relevantes no *recall*. Em aplicações de segurança cibernética, esse comportamento é desejável, visto que a prioridade recai sobre a capacidade de identificar o maior número possível de ataques, ainda que isso possa resultar em leves aumentos de falsos positivos. Essa interpretação encontra suporte em trabalhos como Adewole (2025), que enfatizam a necessidade de privilegiar a sensibilidade em sistemas de IDS.

Outro aspecto fundamental foi a integração do pipeline otimizado com *Apache Kafka* e *Apache Flink*. A arquitetura implementada demonstrou viabilidade na execução contínua de dados e na análise em fluxo, confirmando o potencial de implantação prática em ambientes reais. Trabalhos como Waehner (2022) reforçam a relevância dessa abordagem, destacando como pipelines de inferência integrados a sistemas de *streaming* são essenciais no contexto atual de detecção de ameaças, fraudes e anomalias.

Ao comparar os resultados deste estudo com aqueles apresentados na literatura, observa-se que, embora os valores de acurácia obtidos estejam próximos aos relatados em estudos como Yakub (2025), a métrica de *F1-score* obtida permanece inferior aos valores alcançados por pesquisas como Çoşkun e Çetin (2022) e Gou (2023), que reportam desempenhos significativamente mais elevados, especialmente em classes minoritárias. Essa diferença evidencia a forte influência do desbalanceamento do conjunto CICIDS2017 e reforça a necessidade de empregar estratégias adicionais de tratamento de classes, como *SMOTE* e abordagens *cost-sensitive learning*, amplamente utilizadas nos trabalhos que obtiveram resultados superiores.

Apesar dessa diferença, observa-se que o presente estudo se distingue por seu foco operacional e pela proposta de um *pipeline* completo de inferência otimizado para tempo real, elemento que não é tratado em profundidade nos estudos comparados. Enquanto trabalhos como Çoşkun e Çetin (2022) e Yakub (2025) enfatizam o desempenho estático dos modelos em termos de métricas, este estudo avança ao demonstrar como esses modelos podem ser integrados, otimizados e executados em uma arquitetura moderna de *streaming*, evidenciando uma contribuição prática relevante e alinhada às necessidades de ambientes corporativos e sistemas distribuídos.

Além disso, o fato de os modelos otimizados via *ONNX Runtime* apresentarem maior eficiência computacional e menor latência media reforça a aplicabilidade da abordagem em sistemas que exigem respostas rápidas e escalabilidade. Essa constatação vai ao encontro dos relatos presentes em Microsoft (2023), que apontam ganhos significativos de desempenho com o uso do *runtime* em inferência sobre CPU, especialmente quando associado a modelos baseados em árvores de decisão.

Conclui-se, portanto, que a abordagem proposta representa uma alternativa viável, eficiente e compatível com os requisitos contemporâneos de detecção de intrusões em larga escala. Embora exista espaço para avanços no tratamento do desbalanceamento e no aumento do *F1-score* em classes minoritárias, o *pipeline* construído demonstrou ser tecnicamente sólido, escalável e adequado para ambientes reais de monitoramento de rede. Dessa forma, o trabalho contribui tanto para o avanço científico quanto para a aplicação prática de soluções inteligentes voltadas à segurança cibernética, consolidando-se como uma base para futuras investigações e aprimoramentos em sistemas inteligentes de proteção de redes.

Referências

ADEWOLE, e. a. Intrusion detection framework for internet of things with ensemble learning and rule induction. *Sensors*, v. 25, n. 6, 2025. Citado 5 vezes nas páginas 45, 47, 53, 54 e 55.

ALVES, A.; CARVALHO, F.; FONSECA, J. Big iot: Uma aplicação para processamento de dados da iot na era do big data. In: *Anais da Jornada Científica e de Extensão (JCE 2019) - Resumos Estendidos, Semana Universitária (SUPER 2019)*. [s.n.], 2019. Acesso em: 17 maio 2025. Disponível em: <http://www.pachecoj.com/courses/csc380_fall21/lectures/mlintro.pdf>. Citado na página 33.

ANJI. *Apache Kafka Fundamentals*. 2019. Acesso em: 05 junho 2025. Disponível em: <https://medium.com/techmonks/apache-kafka-terminology-29ca3b27c111>. Citado 2 vezes nas páginas 34 e 35.

ANONYMOUS. *Analysis of CICIDS2017 Dataset Issues and Fixes*. 2023. Disponível em: <https://arxiv.org/pdf/2310.01850>. Citado na página 26.

AUTENTIFY. *Ia Fraca E Ia Forte: Entenda Suas Diferenças Essenciais*. 2024. Acesso em: 22 maio 2025. Disponível em: <https://www.autentify.com.br/inteligencia-artificial/ia-fraca-e-ia-forte-entenda-suas-diferencas-essenciais/>. Citado na página 26.

AWS. *Qual é a diferença entre IPv4 e IPv6?* 2022. Acessado em: 23 maio 2025. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-ipv4-and-ipv6/>. Citado 2 vezes nas páginas 19 e 20.

BBC. *Second bank cyber-attack detected by Swift after Bangladesh raid*. 2016. Acesso em: 04 de setembro 2025. Disponível em: <https://www.bbc.com/news/technology-36284446>. Citado na página 12.

BRAGA, P. H. S. *IMPLANTAÇÃO DE MODELOS DE APRENDIZADO DE MÁQUINA NO FORMATO ONNX UTILIZANDO DIFERENTES FRAMEWORKS*. 2023. Acesso em: 10 outubro 2025. Citado na página 37.

BRUYNE, P. de; HERMAN, J.; SCHOUTHEETE, M. de. *Dinâmica da Pesquisa em Ciências Sociais: os polos da prática metodológica*. Rio de Janeiro: Francisco Alves, 1977. Citado na página 14.

BYTEBYTEGO. *How PayPal Scaled Kafka to 1.3 Trillion Daily Messages*. 2024. Acesso em: 28 maio de 2025. Disponível em: <https://blog.bytebytego.com/p/how-paypal-scaled-kafka-to-13-trillion>. Citado na página 11.

Canadian Institute for Cybersecurity. *CICIDS2017 - Intrusion Detection Evaluation Dataset*. 2017. Acesso em: 22 maio 2025. Disponível em: <https://www.unb.ca/cic/datasets/ids-2017.html>. Citado na página 24.

CARVALHO PAULO E SILVA, D. *Investigação sobre Técnicas de Detecção de Intrusões em Redes de Computadores com base nos Algoritmos Knn e K-Means*. 2015. Acesso em: 01 maio 2025. Disponível em: <https://www.researchgate.net/publication/283325462>. Citado na página 11.

CHEN, C. G. T. *XGBoost: A Scalable Tree Boosting System*. 2015. Acesso em = 21 agosto 2025. Citado 2 vezes nas páginas 12 e 28.

COULOURIS, G. et al. *Sistemas Distribuídos: Conceitos e Projeto*. 5. ed. [S.l.]: Bookman, 2012. Citado na página 24.

ENGELEN, K. *Troubleshooting the CICIDS2017 dataset*. 2021. Workshop on Traffic Measurements for Cybersecurity (WTMC). Disponível em: <https://intrusion-detection.distrinet-research.be/WTMC2021/Resources/wtmc2021_Engelen_Troubleshooting.pdf>. Citado 2 vezes nas páginas 25 e 26.

ESMAEILI, A. et al. *Insights into modeling refractive index of ionic liquids using chemical structure-based machine learning methods*. 2023. Acesso em: 22 maio 2025. Citado 2 vezes nas páginas 30 e 31.

FERREIRA, V. A. *Sistema de detecção de intrusão baseado em fluxo de dados utilizando aprendizado de máquina*. 2017. Acesso em: 01 maio 2025. Disponível em: <https://repositorio.ufmg.br/handle/1843/SLSC-BC9F4H>. Citado na página 27.

FLINK, A. *Apache Flink Documentation*. 2024. Acesso em: 17 maio 2025. Disponível em: <https://nightlies.apache.org/flink//flink-docs-release-1.17/>. Citado na página 33.

FLOREK, P.; ZAGDAńSKI, A. *Benchmarking state-of-the-art gradient boosting algorithms for classification*. 2023. Acesso em: 01 maio 2025. Disponível em: <https://arxiv.org/abs/2305.17094>. Citado 3 vezes nas páginas 11, 30 e 33.

FORUM, B. P. *Fundamentos de Roteamento para Provedores*. 2020. Acesso em: 04 de setembro 2025. Disponível em: <https://wiki.brasilpeeringforum.org/w/Fundamentos_de_Roteamento_para_Provedores>. Citado na página 19.

GIL, A. C. *Métodos e técnicas de pesquisa social*. São Paulo: Atlas, 2008. Citado na página 42.

GOU, e. a. Multi-classification and tree-based ensemble network for intrusion detection systems using cicids2017. *Sensors*, v. 23, n. 21, p. 8788, 2023. Citado 2 vezes nas páginas 54 e 55.

IBM. *Wide Area Network (WAN)*. 2023. Acesso em: 22 maio 2025. Disponível em: <https://www.ibm.com/think/topics/wide-area-network>. Citado 2 vezes nas páginas 22 e 23.

ILERI, e. a. Comparative analysis of catboost, lightgbm, xgboost, rf and dt methods optimised with pso. *Wireless Networks*, 2025. Citado 3 vezes nas páginas 45, 51 e 53.

KAFKA, A. *Apache Kafka Documentation*. 2024. Acesso em: 17 maio 2025. Disponível em: <https://kafka.apache.org/documentation/>. Citado na página 33.

KAPADIA, M. et al. *Data Visualization and Preprocessing for Network Traffic Analysis in Cybersecurity*. 2025. Disponível em: <https://iecscience.org/uploads/jpapers/202508/HPbnimrIhntWAtBMZO8IcmKt6tnIWykNiA192qde.pdf>. Citado na página 42.

KE, G. et al. Lightgbm: A highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. Acesso em: 25 abril 2025. Disponível em: <https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradientboosting-decision-tree.pdf>. Citado 4 vezes nas páginas 30, 31, 32 e 41.

KUROSE, J. F.; ROSS, K. W. *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. 6. ed. [S.l.]: Pearson, 2021. Citado 3 vezes nas páginas 18, 20 e 21.

LIANG, T. et al. Pruning and quantization for deep neural network acceleration: A survey. *arXiv preprint*, 2021. Acesso em: 01 maio 2025. Disponível em: <https://arxiv.org/abs/2101.09671>. Citado 4 vezes nas páginas 11, 36, 38 e 41.

MARIANO, D. *Métricas de avaliação em machine learning*. 2021. Acesso em: 20 maio 2025. Citado 2 vezes nas páginas 39 e 41.

MICROSOFT. 2022. Acesso em 10 outubro de 2025. Disponível em: <https://developer.nvidia.com/blog/end-to-end-ai-for-pcs-onnx-runtime-and-optimization/>. Citado 2 vezes nas páginas 11 e 12.

MICROSOFT. *ONNX Runtime Performance Tuning Guide*. 2023. Disponível em: <https://onnxruntime.ai/>. Citado 2 vezes nas páginas 53 e 56.

MOHAMMED, M.; KADHEM, S.; MAISA. *Insider Attacker Detection Based On Body Language and Technical Behavior Using Light Gradient Boosting Machine (LightGBM)*. 2021. Acesso em: 22 maio 2025. Citado na página 32.

NVIDIA. *NVIDIA TensorRT Model Optimizer Boosts Inference Performance and Expands Model Support*. 2024. Acesso em: 01 maio 2025. Disponível em: <https://developer.nvidia.com/blog/nvidia-tensorrt-model-optimizer-v0-15-boosts-inference-performance-and-expands-model-support/>. Citado na página 36.

NVIDIA. *Sparsity in INT8: Training Workflow and Best Practices for NVIDIA TensorRT Acceleration*. 2025. Acesso em: 25 abril 2025. Disponível em: <https://resources.nvidia.com/en-us-ai-inference-content/sparsity-in-int8-tra>. Citado 3 vezes nas páginas 36, 38 e 39.

ONE, C. *Exploring Apache Flink AWS KDA: Realtime data streaming*. 2025. Acesso em: 28 maio de 2025. Disponível em: <https://www.capitalone.com/tech/cloud/aws-apache-flink/>. Citado na página 11.

PACHECO, J. *Introduction to Machine Learning*. 2021. Acesso em: 17 maio 2025. Disponível em: <http://www.pachecoj.com/courses/csc380_fall21/lectures/mlintro.pdf>. Citado na página 27.

PAES, D. S. F. *Detecção preditiva de anomalias em redes de computadores com utilização de aprendizagem de máquina*. 2023. Acesso em: 19 de junho 2025. Citado 2 vezes nas páginas 20 e 21.

PAN, G. J. *Ciberataque compromete 76 milhões de contas do banco americano JPMorgan Chase*. 2014. Acesso em: 04 de setembro 2025. Disponível em: <https://jovempan.com.br/noticias/

ciberataque-compromete-76-milhoes-de-contas-do-banco-americano-jpmorgan-chase-2014-10-02.html>. Citado na página 12.

PATCHIPALA, S. G. Real-time ai analytics with apache flink: Powering immediate insights with stream processing. *World Journal of Advanced Engineering Technology and Sciences*, v. 13, n. 2, p. 038–050, 2024. Acesso em: 29 abril 2025. Disponível em: <https://doi.org/10.30574/wjaets.2024.13.2.0539>. Citado 2 vezes nas páginas 35 e 36.

PEFFERS, K. et al. A design science research methodology for information systems research. *Journal of Management Information Systems*, M.E. Sharpe, v. 24, n. 3, p. 45–77, 2007. Citado na página 14.

PINHEIRO, J. M. H. Um estudo sobre algoritmos de boosting e a otimização de hiperparâmetros utilizando optuna. 2023. Disponível em: <https://bdta.abcd.usp.br/directbitstream/6962846b-66bd-4bd6-9f74-2b18bff03234/Pinheiro_Jo%C3%A3oManoelHerrera_tcc.pdf>. Citado na página 28.

PRODANOV, C. C.; FREITAS, E. C. de. *Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico*. 2. ed. Novo Hamburgo: Feevale, 2013. Citado na página 14.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3. ed. [S.l.]: Prentice Hall, 2010. ISBN 978-0136042594. Citado na página 26.

SIGMA. *O que é rede WAN e rede LAN?* 2022. Acesso em: 23 de maio 2025. Citado 2 vezes nas páginas 22 e 23.

SMITH, J.; LEE, D. Enhancing intrusion detection: A hybrid machine and deep learning model using xgboost-lstm evaluated on cicids2017. *Journal of Cloud Computing*, 2024. Hybrid XGBoost + LSTM model. Citado na página 25.

SOUMIK, S. Survey of cicids2017 dataset for machine learning-based ids. *International Journal of Scientific Research in Advances*, 2024. Disponível em: <https://ijsra.net/sites/default/files/IJSRA-2024-2664.pdf>. Citado na página 25.

SOUZA, J.; SALLES, R. *Aprendizagem de Máquina para Segurança em Redes de Computadores: Métodos e Aplicações*. 2011. Acesso em: 26 abril 2025. Disponível em: <https://www.researchgate.net/publication/228447003>. Citado na página 11.

SYSTEMS, C. *What Is a LAN (Local Area Network)?* 2023. Acesso em: 22 maio 2025. Disponível em: <https://www.cisco.com/c/en_sg/products/switches/what-is-a-lan-local-area-network.html>. Citado na página 21.

TANENBAUM, A. S.; WETHERALL, D. J. *Redes de Computadores*. 5. ed. São Paulo: Pearson Education, 2011. Citado 4 vezes nas páginas 18, 19, 20 e 21.

TANG, e. a. An efficient intrusion detection method based on lightgbm and autoencoder. *Symmetry*, v. 12, n. 9, 2020. Citado 4 vezes nas páginas 45, 46, 47 e 53.

TOMIO, R. L. *Análise comparativa de algoritmos de aprendizado de máquina aplicados à detecção de intrusões em redes*. Dissertação (Mestrado) — PUC Paraná, Curitiba, 2021. Citado 3 vezes nas páginas 46, 49 e 54.

TOMIO, R. L. *UM MODELO BASEADO EM APRENDIZAGEM DE MÁQUINA PARA DETECÇÃO AUTÔNOMA DE INTRUSÃO*. 2021. Acesso em: 27 maio de 2025. Disponível em: <https://www.ppgia.pucpr.br/pt/arquivos/mestrado/dissertacoes/2021/Disserta%C3%A7%C3%A3o_Rivaldo_Luiz_Tomio.pdf>. Citado na página 41.

TÜBİTAK. A review of cicids2017 for intrusion detection models. *TÜBİTAK Journal of Electrical Engineering & Computer Sciences*, 2023. Disponível em: <https://journals.tubitak.gov.tr/elektrik>. Citado na página 25.

WAEHNER, K. *Fraud Detection with Apache Kafka and Apache Flink*. 2022. Acesso em: 25 abril 2025. Disponível em: <https://www.kai-waehner.de/blog/2022/10/25/fraud-detection-with-apache-kafka-ksql-and-apache-flink/>. Citado 7 vezes nas páginas 12, 34, 36, 41, 46, 53 e 55.

YAKUB, e. a. Intrusion detection framework for internet of things with ensemble learning and rule induction. *Sensors*, v. 25, n. 6, p. 1845, 2025. Citado 2 vezes nas páginas 54 e 55.

ÇOşKUN, A.; ÇETIN, M. A comparative evaluation of the boosting algorithms in network intrusion detection on cicids2017. *Journal of Cybersecurity Research*, 2022. Working paper; Google Colab experiments. Citado 2 vezes nas páginas 54 e 55.

Apêndices

APÊNDICE A – Repositório do Projeto

Todo o código-fonte desenvolvido neste trabalho, incluindo scripts de pré-processamento de dados, treinamento de modelos, inferência em tempo real, arquivos de configuração e instruções de execução, está disponível em um repositório público, garantindo transparência e reprodutibilidade dos experimentos apresentados.

O repositório pode ser acessado por meio do seguinte endereço:

<https://github.com/lucashernandez90/Security-IA>

Nos apêndices subsequentes são apresentados os principais scripts utilizados para a geração dos resultados experimentais discutidos neste trabalho. Os demais arquivos auxiliares, bem como versões atualizadas do sistema, encontram-se disponíveis exclusivamente no repositório.

APÊNDICE B – Script Kafka Producer para Geração de Tráfego

Este apêndice apresenta *script* responsável pela simulação de tráfego de rede e envio de dados ao *Apache Kafka*. O objetivo deste componente é emular um ambiente real de monitoramento contínuo, produzindo mensagens estruturadas contendo características extraídas do tráfego de rede, inspiradas no conjunto de dados CICIDS2017.

O *script* foi implementado em *Python* utilizando a biblioteca *kafka-python* e gera mensagens no formato *JSON*, que são posteriormente consumidas pelo processamento em fluxo no *Apache Flink*.

Listing B.1 – *Kafka Producer* para geração de tráfego

```
1  from kafka import KafkaProducer
2  import json
3  import time
4  import numpy as np
5  from datetime import datetime
6  import logging
7  import argparse
8  import os
9
10 # Configura  o de logging
11 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s -
       %(message)s')
12 logger = logging.getLogger(__name__)
13
14 class NetworkTrafficProducer:
15     def __init__(self, bootstrap_servers: str):
16         self.producer = KafkaProducer(
17             bootstrap_servers=bootstrap_servers,
18             value_serializer=lambda v: json.dumps(v).encode("utf-8"),
19             acks=1,
20             compression_type='gzip',
21             linger_ms=5,
22             retries=3
23         )
24         self.messages_sent = 0
25         self.errors = 0
26         logger.info(f"Producer conectado ao Kafka: {bootstrap_servers}")
27
28     def generate_sample_features(self, num_features: int = 67) -> list:
29         """Gera features de exemplo baseadas no CICIDS2017 com valores realistas"""
30         # Valores t picos do dataset CICIDS2017
31         features = [
32             float(np.random.uniform(0, 1000)),        # Flow Duration
33             float(np.random.uniform(0, 65535)),       # Destination Port
34             float(np.random.uniform(0, 1000)),        # Total Fwd Packets
35             float(np.random.uniform(0, 1000)),        # Total Backward Packets
36             float(np.random.uniform(0, 1000000)),     # Total Length of Fwd Packets
37             float(np.random.uniform(0, 1000000)),     # Total Length of Bwd Packets
38             float(np.random.uniform(0, 1500)),        # Fwd Packet Length Max
39             float(np.random.uniform(0, 1500)),        # Fwd Packet Length Min
40             float(np.random.uniform(0, 1500)),        # Fwd Packet Length Mean
41             float(np.random.uniform(0, 500)),         # Fwd Packet Length Std
42             float(np.random.uniform(0, 1500)),        # Bwd Packet Length Max
43             float(np.random.uniform(0, 1500)),        # Bwd Packet Length Min
44             float(np.random.uniform(0, 1500)),        # Bwd Packet Length Mean
45             float(np.random.uniform(0, 500)),         # Bwd Packet Length Std
46             float(np.random.uniform(0, 1e9)),         # Flow Bytes/s
47             float(np.random.uniform(0, 1e6)),         # Flow Packets/s
48             float(np.random.uniform(0, 10000)),       # Flow IAT Mean
```

```
49                 float(np.random.uniform(0, 10000)),       # Flow IAT Std
50                 float(np.random.uniform(0, 100000)),      # Flow IAT Max
51                 float(np.random.uniform(0, 1000)),        # Flow IAT Min
52             ]
53
54             # Completar com valores aleat rios para as demais features
55             while len(features) < num_features:
56                 # Gerar valores com distribui  es mais realistas
57                 if np.random.random() < 0.3:  # 30% chance de valor zero
58                     features.append(0.0)
59                 else:
60                     features.append(float(np.random.uniform(0, 1)))
61
62             return features[:num_features]
63
64         def generate_from_real_data(self, data_path: str = None):
65             """Gera features a partir de dados reais se dispon vel"""
66             if data_path and os.path.exists(data_path):
67                 try:
68                     # Carregar amostra real para maior realismo
69                     real_data = np.load(data_path)
70                     if len(real_data) > 0:
71                         sample = real_data[np.random.randint(0, len(real_data))]
72                         return sample.tolist()
73                 except Exception as e:
74                     logger.warning(f"N o foi poss vel carregar dados reais: {e}")
75
76             # Fallback para gera  o sint tica
77             return self.generate_sample_features()
78
79         def send_message(self, topic: str, message: dict):
80             try:
81                 future = self.producer.send(topic, value=message)
82                 future.get(timeout=10)
83                 self.messages_sent += 1
84                 if self.messages_sent % 100 == 0:
85                     logger.info(f"Mensagens enviadas: {self.messages_sent}")
86                 return True
87             except Exception as e:
88                 self.errors += 1
89                 logger.error(f"Erro ao enviar mensagem {message.get('id', 'N/A')}: {e}")
90                 return False
91
92         def close(self):
93             self.producer.flush()
94             self.producer.close()
95             logger.info(f"Producer fechado. Estat sticas: {self.messages_sent} mensagens
                   enviadas, {self.errors} erros")
96
97     def main():
98         parser = argparse.ArgumentParser(description='Kafka Producer para tr fego de rede')
99         parser.add_argument('--bootstrap-servers', default='localhost:9092', help='Endere os
               dos brokers Kafka')
100        parser.add_argument('--topic', default='network-traffic', help='T pico Kafka')
101        parser.add_argument('--num-messages', type=int, default=100, help='N mero total de
               mensagens')
102        parser.add_argument('--delay', type=float, default=0.1, help='Delay entre mensagens
               em segundos')
103        parser.add_argument('--real-data', type=str, help='Caminho para arquivo .npy com
               dados reais')
104        parser.add_argument('--batch-size', type=int, default=10, help='Tamanho do lote para
               logging')
105        args = parser.parse_args()
106
107        producer = NetworkTrafficProducer(args.bootstrap_servers)
108
109        try:
110            start_time = time.time()
111
112            for i in range(args.num_messages):
113                # Gerar features (com dados reais se dispon vel)
114                if args.real_data:
115                    features = producer.generate_from_real_data(args.real_data)
```

```
116             else:
117                 features = producer.generate_sample_features()
118 
119             message = {
120                 "id": i,
121                 "timestamp": datetime.utcnow().isoformat() + "Z",
122                 "features": features,
123                 "source_ip": f"192.168.{np.random.randint(1, 255)}.{np.random.randint(1,
                        255)}",
124                 "destination_ip": f"10.{np.random.randint(0, 255)}.{np.random.randint(0,
                        255)}.{np.random.randint(1, 255)}",
125                 "destination_port": int(np.random.uniform(0, 65535)),
126                 "protocol": np.random.choice(["TCP", "UDP", "ICMP"]),
127                 "prediction": None,  # Para ser preenchido pelo consumer
128                 "confidence": None   # Para ser preenchido pelo consumer
129             }
130 
131             success = producer.send_message(args.topic, message)
132 
133             if not success and args.delay > 0:
134                 # Aumentar delay temporariamente em caso de erro
135                 time.sleep(args.delay * 2)
136             else:
137                 time.sleep(args.delay)
138 
139             # Log a cada batch
140             if i % args.batch_size == 0 and i > 0:
141                 elapsed = time.time() - start_time
142                 rate = i / elapsed if elapsed > 0 else 0
143                 logger.info(f"Progresso: {i}/{args.num_messages} mensagens | Rate: {rate
                        :.2f} msg/s")
144 
145     except KeyboardInterrupt:
146         logger.info("Interrup  o recebida pelo usu rio")
147     except Exception as e:
148         logger.error(f"Erro no producer: {e}")
149     finally:
150         producer.close()
151 
152         # Estat sticas finais
153         total_time = time.time() - start_time
154         logger.info(f"Tempo total: {total_time:.2f}s")
155         if total_time > 0:
156             logger.info(f"Taxa m dia: {args.num_messages/total_time:.2f} msg/s")
157 
158 if __name__ == "__main__":
159     main()
```

APÊNDICE C – Job Apache Flink para Detecção de Intrusões

Este apêndice apresenta o *job* implementado no *Apache Flink* responsável pela leitura contínua das mensagens provenientes do *Apache Kafka*, aplicação do modelo de aprendizado de máquina e geração das predições de intrusão em tempo real.

O modelo treinado foi exportado para o formato ONNX e executado por meio do *ONNX Runtime*, permitindo inferência eficiente em ambiente de produção, independente de hardware especializado.

Listing C.1 – *Job Flink* com inferência ONNX

```
# src/streaming/advanced_processor.py
import json
import time
import logging
from kafka import KafkaConsumer, KafkaProducer
import requests

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def continuous_processor():
    """Processador CONTNUO que fica rodando indefinidamente"""

    logger.info("PROCESSOR INICIADO!")
    logger.info("Consumindo: network-traffic")
    logger.info("Produzindo: ids-predictions")
    logger.info("API: http://localhost:5005")
    logger.info("Pressione CTRL+C para parar")

    # Consumer SEM timeout - fica rodando para sempre
    consumer = KafkaConsumer(
        'network-traffic',
        bootstrap_servers='localhost:9092',
        auto_offset_reset='latest',  # S   mensagens novas
        enable_auto_commit=False,
        group_id=None,  # Sem grupo
        value_deserializer=lambda x: json.loads(x.decode('utf-8'))
        # SEM consumer_timeout_ms - fica esperando indefinidamente
    )

    producer = KafkaProducer(
        bootstrap_servers='localhost:9092',
        value_serializer=lambda x: json.dumps(x).encode('utf-8')
    )

    # Verificar API
    try:
        response = requests.get('http://localhost:5005/health', timeout=3)
        logger.info(f"  API dispon vel: {response.status_code}")
    except Exception as e:
        logger.error(f" API n o dispon vel: {e}")
        return

    message_count = 0

    try:
        logger.info("   Aguardando mensagens... (Envie com producer.py)")

        # Loop infinito - processa mensagens continuamente
        for message in consumer:
            message_count += 1
```

```
52              logger.info(f"  MENSAGEM #{message_count} RECEBIDA!")
53              logger.info(f"   Offset: {message.offset}")
54
55              data = message.value
56              logger.info(f"  Tipo de dados: {type(data)}")
57
58              # Extrair features
59              features = []
60              if isinstance(data, dict) and 'features' in data:
61                  features = data['features'][:66]  # Primeiras 66 features
62                  logger.info(f"  {len(features)} features extra das")
63              else:
64                  features = [0.1] * 66
65                  logger.warning("Usando features padr o")
66
67              # Chamar API
68              logger.info("Chamando API de detec   o...")
69              try:
70                  response = requests.post(
71                      'http://localhost:5005/detect_extreme',
72                      json={"features": features},
73                      timeout=5
74                  )
75
76                  if response.status_code == 200:
77                      result = response.json()
78                      prediction = result.get("prediction", "UNKNOWN")
79                      confidence = result.get("confidence", 0)
80
81                      # Criar mensagem de prediction
82                      prediction_msg = {
83                          "message_id": message_count,
84                          "timestamp": time.time(),
85                          "prediction": prediction,
86                          "confidence": round(confidence, 3),
87                          "offset": message.offset,
88                          "status": "processed"
89                      }
90
91                      # Enviar para Kafka
92                      producer.send('ids-predictions', prediction_msg)
93                      producer.flush()
94
95                      # Log colorido
96                      if prediction == "BENIGN":
97                          logger.info(f" <—— #{message_count}: {prediction} (conf: {
                                confidence:.3f}) ——>" )
98                      else:
99                          logger.error(f" >>>>>>> #{message_count}: {prediction} (conf: {
                                confidence:.3f}) <<<<<<<")
100
101                     logger.info(f"Prediction enviada para ids-predictions")
102
103                 else:
104                     logger.error(f"API Error: {response.status_code}")
105
106             except Exception as e:
107                 logger.error(f"Erro na chamada da API: {e}")
108
109             logger.info("=" * 50)
110             logger.info("Aguardando pr xima mensagem...")
111
112     except KeyboardInterrupt:
113         logger.info("   Processador interrompido pelo usu rio")
114     except Exception as e:
115         logger.error(f" Erro fatal: {e}")
116     finally:
117         consumer.close()
118         producer.close()
119         logger.info(f"Finalizado. Total: {message_count} mensagens processadas")
120
121 if __name__ == "__main__":
122     continuous_processor()
```

APÊNDICE D – Treinamento e Avaliação dos Modelos

Este apêndice apresenta o *script* utilizado para o treinamento e avaliação dos modelos de aprendizado de máquina empregados neste trabalho, incluindo *XGBoost* e *LightGBM*. O treinamento foi realizado a partir do conjunto de dados CICIDS2017, após as etapas de limpeza, normalização e seleção de atributos.

São definidos explicitamente os hiperparâmetros utilizados, o processo de divisão dos dados em conjuntos de treino e teste, bem como as métricas adotadas para avaliação do desempenho.

Listing D.1 – Treinamento e avaliação dos modelos

```
"""
train.py      Treina XGBoost e LightGBM usando os .npy de data/processed.
Exemplo:
    python src/train.py --in-dir data/processed --out-models models
    --out-reports reports --task multiclass --train-iterations 3
"""
from __future__ import annotations
import argparse
import json
import os
import joblib
import numpy as np
import logging
import shutil
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
from utils import ensure_dir, save_json, compute_metrics, plot_confusion_matrix

# Configura  o de logging
logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

def load_processed(in_dir: str):
    """Carrega dados pr -processados + metadados"""
    try:
        # Carregar dados
        X_train = np.load(os.path.join(in_dir, "X_train_balanced.npy"))
        X_test = np.load(os.path.join(in_dir, "X_test_scaled.npy"))
        y_train = np.load(os.path.join(in_dir, "y_train_balanced.npy"))
        y_test = np.load(os.path.join(in_dir, "y_test.npy"))

        # Carregar encoder para obter nomes reais das classes
        le = joblib.load(os.path.join(in_dir, "label_encoder.pkl"))

        # Carregar metadata
        metadata_path = os.path.join(in_dir, "metadata.json")
        if os.path.exists(metadata_path):
            meta = json.load(open(metadata_path, encoding="utf-8"))
            # CORRE O: Garantir que n_classes existe
            if 'n_classes' not in meta:
                meta['n_classes'] = len(le.classes_)
        else:
            # Criar metadata automaticamente com nomes reais
            meta = {
                "class_names": le.classes_.tolist(),
                "n_classes": len(le.classes_),
```

```
50                  "input_shape": X_train.shape[1:]
51              }
52
53          logger.info(f"Dados carregados: X_train={X_train.shape}, X_test={X_test.shape}")
54          logger.info(f"N mero de classes: {meta['n_classes']}")
55          logger.info(f"Nomes das classes: {meta['class_names']}")
56
57          # Log da distribui  o
58          unique_train, counts_train = np.unique(y_train, return_counts=True)
59          unique_test, counts_test = np.unique(y_test, return_counts=True)
60
61          logger.info("Distribui  o treino:")
62          for cls, count in zip(unique_train, counts_train):
63              class_name = le.inverse_transform([cls])[0]
64              logger.info(f"  {class_name}: {count:,}")
65
66          logger.info("Distribui  o teste:")
67          for cls, count in zip(unique_test, counts_test):
68              class_name = le.inverse_transform([cls])[0]
69              logger.info(f"  {class_name}: {count:,}")
70
71          return X_train, X_test, y_train, y_test, meta, le
72
73      except Exception as e:
74          logger.error(f"Erro ao carregar dados processados: {e}")
75          raise
76
77  def train_xgboost(X_train, y_train, task: str):
78      """Treina modelo XGBoost"""
79      try:
80          from xgboost import XGBClassifier
81          params = dict(
82              n_estimators=200,
83              learning_rate=0.1,
84              max_depth=6,
85              subsample=0.8,
86              colsample_bytree=0.8,
87              reg_alpha=0.0,
88              reg_lambda=1.0,
89              random_state=42,
90              n_jobs=-1,
91              tree_method="hist",
92              eval_metric="mlogloss" if task == "multiclass" else "logloss",
93              verbosity=0
94          )
95
96          if task == "binary":
97              pos, neg = (y_train == 1).sum(), (y_train == 0).sum()
98              if pos > 0:
99                  params["scale_pos_weight"] = neg / pos
100                 logger.info(f"Scale pos weight calculado:
                        {params['scale_pos_weight']:.2f}")
101         else:
102             # Para multiclass
103             params["objective"] = "multi:softprob"
104             params["num_class"] = len(np.unique(y_train))
105
106         model = XGBClassifier(**params)
107         model.fit(X_train, y_train)
108         logger.info("XGBoost treinado com sucesso")
109         return model, params
110     except Exception as e:
111         logger.error(f"Erro no treinamento XGBoost: {e}")
112         raise
113
114 def train_lightgbm(X_train, y_train, task: str):
115     """Treina modelo LightGBM"""
116     try:
117         from lightgbm import LGBMClassifier
118         params = dict(
119             n_estimators=200,
120             learning_rate=0.05,
121             max_depth=-1,
```

```
            num_leaves=31,
            subsample=0.8,
            colsample_bytree=0.8,
            reg_alpha=0.0,
            reg_lambda=1.0,
            random_state=42,
            n_jobs=-1,
            verbose=-1
        )

        if task == "binary":
            params["objective"] = "binary"
            params["is_unbalance"] = True
        else:
            params["objective"] = "multiclass"
            params["num_class"] = len(np.unique(y_train))

        model = LGBMClassifier(**params)
        model.fit(X_train, y_train)
        logger.info("LightGBM treinado com sucesso")
        return model, params
    except Exception as e:
        logger.error(f"Erro no treinamento LightGBM: {e}")
        raise

def kfold_cv(model_ctor, X, y, task: str, n_splits: int = 3):
    """Executa valida  o cruzada estratificada otimizada"""
    try:
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
        scores = []

        logger.info(f"Iniciando {n_splits}-fold cross validation...")

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
            logger.info(f"Processando fold {fold + 1}/{n_splits}")

            X_fold_train, y_fold_train = X[train_idx], y[train_idx]
            X_fold_val, y_fold_val = X[val_idx], y[val_idx]

            # Amostrar apenas se necess rio para velocidade
            if len(X_fold_train) > 50000:
                from sklearn.utils import resample
                X_fold_train, y_fold_train = resample(
                    X_fold_train, y_fold_train,
                    n_samples=50000, random_state=42, stratify=y_fold_train
                )
                logger.info(f"  Fold {fold + 1}: Dataset reduzido para treino")

            # Treinar modelo
            m, _ = model_ctor(X_fold_train, y_fold_train, task)

            # Avaliar
            y_pred = m.predict(X_fold_val)
            avg = "binary" if task == "binary" else "macro"
            s = compute_metrics(y_fold_val, y_pred, average=avg)
            scores.append({k: float(s[k]) for k in ["accuracy", "precision", "recall",
                "f1"]})

            logger.info(f"  Fold {fold + 1}: Accuracy={s['accuracy']:.4f},
                F1={s['f1']:.4f}")

        # Calcular m dias
        agg_scores = {}
        for metric in ["accuracy", "precision", "recall", "f1"]:
            values = [fold[metric] for fold in scores]
            agg_scores[metric] = float(np.mean(values))
            agg_scores[f"{metric}_std"] = float(np.std(values))

        logger.info(f"CV Final - Accuracy: {agg_scores['accuracy']:.4f}
            (  {agg_scores['accuracy_std']:.4f})")
        logger.info(f"CV Final - F1: {agg_scores['f1']:.4f}
            (  {agg_scores['f1_std']:.4f})")

```

```
        return {"cv_metrics": agg_scores, "folds": scores}

    except Exception as e:
        logger.error(f"Erro na valida  o cruzada: {e}")
        raise

def save_model_artifacts(name, model, params, cv, X_test, y_test, labels, avg, le,
    out_models, out_reports):
    """Salva modelo, m tricas e relat rios"""
    try:
        from sklearn.metrics import classification_report

        # Predi  es
        y_pred = model.predict(X_test)
        y_prob = model.predict_proba(X_test) if hasattr(model, 'predict_proba') else None
        test_metrics = compute_metrics(y_test, y_pred, average=avg,
            labels=np.arange(len(labels)))

        # Modelo
        model_path = os.path.join(out_models, f"{name}.joblib")
        joblib.dump(model, model_path)
        logger.info(f"Modelo salvo: {model_path}")

        # Matriz de confus o com nomes reais
        cm_path = os.path.join(out_reports, f"cm_{name}.png")
        plot_confusion_matrix(y_test, y_pred, labels,
                              f"{name}     Matriz de Confus o",
                              cm_path)
        logger.info(f"Matriz de confus o salva: {cm_path}")

        # JSON com hiperpar metros e m tricas
        metrics_data = {
            "params": params,
            "cv": cv,
            "test": {k: v for k, v in test_metrics.items() if k !=
                "classification_report"},
            "test_probabilities": {
                "mean_confidence": float(np.mean(np.max(y_prob, axis=1))) if y_prob is
                    not None else None
            },
            "labels": labels,
            "model_info": {
                "n_features": X_test.shape[1],
                "n_test_samples": len(y_test),
                "feature_importances": model.feature_importances_.tolist() if
                    hasattr(model, 'feature_importances_') else None
            }
        }

        metrics_path = os.path.join(out_reports, f"metrics_{name}.json")
        save_json(metrics_data, metrics_path)
        logger.info(f"M tricas salvas: {metrics_path}")

        # Relat rio detalhado
        report_path = os.path.join(out_reports, f"report_{name}.txt")
        with open(report_path, "w", encoding="utf-8") as f:
            f.write(test_metrics["classification_report"])
        logger.info(f"Relat rio salvo: {report_path}")

        return test_metrics, y_pred, y_prob

    except Exception as e:
        logger.error(f"Erro ao salvar artefatos do modelo {name}: {e}")
        raise

def create_comparison_report(all_xgb_metrics, all_lgb_metrics, X_test, y_test, labels,
    le, out_reports):
    """Cria relat rios comparativos completos entre XGBoost e LightGBM"""
    try:
        logger.info("Criando relat rios comparativos...")

        # Encontrar melhores itera  es
        xgb_best_idx = np.argmax([m["accuracy"] for m in all_xgb_metrics])
```

```
        lgb_best_idx = np.argmax([m["accuracy"] for m in all_lgb_metrics])

        best_xgb_acc = all_xgb_metrics[xgb_best_idx]["accuracy"]
        best_lgb_acc = all_lgb_metrics[lgb_best_idx]["accuracy"]
        best_xgb_f1 = all_xgb_metrics[xgb_best_idx]["f1"]
        best_lgb_f1 = all_lgb_metrics[lgb_best_idx]["f1"]

        # ==========================================================
        # 1. RELATRIO COMPARATIVO EM JSON
        # ==========================================================
        comparison_data = {
            "summary": {
                "best_overall_model": "xgboost" if best_xgb_acc > best_lgb_acc else "lightgbm",
                "training_date": str(np.datetime64('now')),
                "total_iterations": len(all_xgb_metrics)
            },
            "performance_comparison": {
                "xgboost": {
                    "best_accuracy": float(best_xgb_acc),
                    "best_f1": float(best_xgb_f1),
                    "best_iteration": int(xgb_best_idx + 1),
                    "average_accuracy": float(np.mean([m["accuracy"] for m in all_xgb_metrics])),
                    "average_f1": float(np.mean([m["f1"] for m in all_xgb_metrics])),
                    "std_accuracy": float(np.std([m["accuracy"] for m in all_xgb_metrics])),
                    "std_f1": float(np.std([m["f1"] for m in all_xgb_metrics]))
                },
                "lightgbm": {
                    "best_accuracy": float(best_lgb_acc),
                    "best_f1": float(best_lgb_f1),
                    "best_iteration": int(lgb_best_idx + 1),
                    "average_accuracy": float(np.mean([m["accuracy"] for m in all_lgb_metrics])),
                    "average_f1": float(np.mean([m["f1"] for m in all_lgb_metrics])),
                    "std_accuracy": float(np.std([m["accuracy"] for m in all_lgb_metrics])),
                    "std_f1": float(np.std([m["f1"] for m in all_lgb_metrics]))
                }
            },
            "detailed_iterations": {
                "xgboost": [
                    {
                        "iteration": i + 1,
                        "accuracy": float(metrics["accuracy"]),
                        "precision": float(metrics["precision"]),
                        "recall": float(metrics["recall"]),
                        "f1": float(metrics["f1"])
                    } for i, metrics in enumerate(all_xgb_metrics)
                ],
                "lightgbm": [
                    {
                        "iteration": i + 1,
                        "accuracy": float(metrics["accuracy"]),
                        "precision": float(metrics["precision"]),
                        "recall": float(metrics["recall"]),
                        "f1": float(metrics["f1"])
                    } for i, metrics in enumerate(all_lgb_metrics)
                ]
            },
            "recommendations": {
                "best_for_accuracy": "xgboost" if best_xgb_acc > best_lgb_acc else "lightgbm",
                "best_for_f1": "xgboost" if best_xgb_f1 > best_lgb_f1 else "lightgbm",
                "most_consistent": "xgboost" if np.std([m["accuracy"] for m in all_xgb_metrics]) < np.std([m["accuracy"] for m in all_lgb_metrics]) else "lightgbm",
                "performance_gap": {
                    "accuracy_difference": float(abs(best_xgb_acc - best_lgb_acc)),
                    "f1_difference": float(abs(best_xgb_f1 - best_lgb_f1))
                }
            }
```

```
        }

        comparison_path = os.path.join(out_reports, "model_comparison.json")
        save_json(comparison_data, comparison_path)
        logger.info(f"Relat rio comparativo JSON salvo: {comparison_path}")

        # ==========================================================
        # 2. GR FICO COMPARATIVO DE PERFORMANCE
        # ==========================================================
        plt.figure(figsize=(15, 10))

        # Dados para plotting
        iterations = range(1, len(all_xgb_metrics) + 1)
        xgb_accuracies = [m["accuracy"] for m in all_xgb_metrics]
        lgb_accuracies = [m["accuracy"] for m in all_lgb_metrics]
        xgb_f1_scores = [m["f1"] for m in all_xgb_metrics]
        lgb_f1_scores = [m["f1"] for m in all_lgb_metrics]

        # Subplot 1: Acur cia por itera   o
        plt.subplot(2, 2, 1)
        plt.plot(iterations, xgb_accuracies, 'o-', label='XGBoost', linewidth=2,
            markersize=8)
        plt.plot(iterations, lgb_accuracies, 's-', label='LightGBM', linewidth=2,
            markersize=8)
        plt.xlabel('Itera   o')
        plt.ylabel('Acur cia')
        plt.title('Acur cia por Itera   o')
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Subplot 2: F1-Score por itera   o
        plt.subplot(2, 2, 2)
        plt.plot(iterations, xgb_f1_scores, 'o-', label='XGBoost', linewidth=2,
            markersize=8)
        plt.plot(iterations, lgb_f1_scores, 's-', label='LightGBM', linewidth=2,
            markersize=8)
        plt.xlabel('Itera   o')
        plt.ylabel('F1-Score')
        plt.title('F1-Score por Itera   o')
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Subplot 3: Compara   o de m tricas (melhor itera   o)
        plt.subplot(2, 2, 3)
        metrics_names = ['Acur cia', 'Precis o', 'Recall', 'F1-Score']
        xgb_best_metrics = [all_xgb_metrics[xgb_best_idx]["accuracy"],
                            all_xgb_metrics[xgb_best_idx]["precision"],
                            all_xgb_metrics[xgb_best_idx]["recall"],
                            all_xgb_metrics[xgb_best_idx]["f1"]]
        lgb_best_metrics = [all_lgb_metrics[lgb_best_idx]["accuracy"],
                            all_lgb_metrics[lgb_best_idx]["precision"],
                            all_lgb_metrics[lgb_best_idx]["recall"],
                            all_lgb_metrics[lgb_best_idx]["f1"]]

        x = np.arange(len(metrics_names))
        width = 0.35

        plt.bar(x - width/2, xgb_best_metrics, width, label='XGBoost', alpha=0.8)
        plt.bar(x + width/2, lgb_best_metrics, width, label='LightGBM', alpha=0.8)
        plt.xlabel('M tricas')
        plt.ylabel('Score')
        plt.title('Compara   o de M tricas (Melhor Itera   o)')
        plt.xticks(x, metrics_names)
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Subplot 4: Resumo estat stico
        plt.subplot(2, 2, 4)
        stats_data = [
            np.mean(xgb_accuracies), np.std(xgb_accuracies),
            np.mean(lgb_accuracies), np.std(lgb_accuracies),
            np.mean(xgb_f1_scores), np.std(xgb_f1_scores),
            np.mean(lgb_f1_scores), np.std(lgb_f1_scores)
```

```
        ]
        stats_labels = ['XGB Acc Mean', 'XGB Acc Std', 'LGB Acc Mean', 'LGB Acc Std',
                        'XGB F1 Mean', 'XGB F1 Std', 'LGB F1 Mean', 'LGB F1 Std']

        colors = ['skyblue', 'lightcoral', 'skyblue', 'lightcoral',
                  'lightgreen', 'lightyellow', 'lightgreen', 'lightyellow']

        bars = plt.barh(stats_labels, stats_data, color=colors)
        plt.xlabel('Valor')
        plt.title('Estat sticas Comparativas')
        plt.grid(True, alpha=0.3)

        # Adicionar valores nas barras
        for bar, value in zip(bars, stats_data):
            plt.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
                     f'{value:.4f}', ha='left', va='center', fontsize=8)

        plt.tight_layout()
        comparison_plot_path = os.path.join(out_reports, "model_comparison.png")
        plt.savefig(comparison_plot_path, dpi=300, bbox_inches='tight')
        plt.close()
        logger.info(f"Gr fico comparativo salvo: {comparison_plot_path}")


        comparison_text_path = os.path.join(out_reports, "model_comparison_report.txt")
        with open(comparison_text_path, "w", encoding="utf-8") as f:
            f.write(comparison_text)
        logger.info(f"Relat rio comparativo em texto salvo: {comparison_text_path}")

        return comparison_data

    except Exception as e:
        logger.error(f"Erro ao criar relat rio comparativo: {e}")
        raise

def main():
    ap = argparse.ArgumentParser(description="Treina modelos de detec  o de intrus o")
    ap.add_argument("--in-dir", default="data/processed", help="Diret rio com dados processados")
    ap.add_argument("--out-models", default="models", help="Diret rio de sa da para modelos")
    ap.add_argument("--out-reports", default="reports", help="Diret rio de sa da para relat rios")
    ap.add_argument("--task", choices=["binary", "multiclass"], default="multiclass", help="Tipo de tarefa de classifica  o")
    ap.add_argument("--cv-folds", type=int, default=3, help="N mero de folds para cross-validation")
    ap.add_argument("--max-train-samples", type=int, default=100000, help="N mero m ximo de amostras para treino")
    ap.add_argument("--train-iterations", type=int, default=3, help="N mero de itera  es de treinamento")
    args = ap.parse_args()

    try:
        ensure_dir(args.out_models)
        ensure_dir(args.out_reports)

        logger.info("Iniciando processo de treinamento...")
        logger.info(f"Par metros: task={args.task}, cv_folds={args.cv_folds}, iterations={args.train_iterations}")

        # Carregar dados com encoder
        X_train, X_test, y_train, y_test, meta, le = load_processed(args.in_dir)
        labels = meta["class_names"]
        avg = "binary" if args.task == "binary" else "macro"

        logger.info(f"N mero de classes: {len(labels)}")
        logger.info(f"Labels: {labels}")

        # Reduzir tamanho do treino se necess rio
        if len(X_train) > args.max_train_samples:
            from sklearn.utils import resample
            X_train, y_train = resample(
```

```
457                 X_train, y_train,
458                 n_samples=args.max_train_samples, random_state=42, stratify=y_train
459             )
460             logger.info(f"Dataset reduzido para: {X_train.shape[0]:,} amostras")
461
462         # Dicion rios para armazenar resultados de todas as itera   es
463         all_xgb_metrics = []
464         all_lgb_metrics = []
465         all_xgb_predictions = []
466         all_lgb_predictions = []
467
468         # ============================================================
469         # TREINAR XGBOOST 3 VEZES COMPLETAS
470         # ============================================================
471         logger.info("=" * 70)
472         logger.info("INICIANDO TREINAMENTO XGBOOST (3 ITERA  ES)")
473         logger.info("=" * 70)
474
475         for iteration in range(args.train_iterations):
476             logger.info("=" * 60)
477             logger.info(f"XGBOOST - ITERA  O {iteration + 1}/{args.train_iterations}")
478             logger.info("=" * 60)
479
480             xgb_cv = kfold_cv(train_xgboost, X_train, y_train, args.task,
                    min(args.cv_folds, 3))
481             xgb_model, xgb_params = train_xgboost(X_train, y_train, args.task)
482             xgb_test_metrics, y_pred, y_prob = save_model_artifacts(
483                 f"xgboost_iter{iteration+1}", xgb_model, xgb_params, xgb_cv,
484                 X_test, y_test, labels, avg, le,
485                 args.out_models, args.out_reports
486             )
487             all_xgb_metrics.append(xgb_test_metrics)
488             all_xgb_predictions.append((y_pred, y_prob))
489
490             logger.info(f"XGBoost Itera   o {iteration + 1} conclu da!")
491             logger.info(f"Acur cia: {xgb_test_metrics['accuracy']:.4f}, F1:
                    {xgb_test_metrics['f1']:.4f}")
492
493         # ============================================================
494         # TREINAR LIGHTGBM 3 VEZES COMPLETAS
495         # ============================================================
496         logger.info("=" * 70)
497         logger.info("INICIANDO TREINAMENTO LIGHTGBM (3 ITERA  ES)")
498         logger.info("=" * 70)
499
500         for iteration in range(args.train_iterations):
501             logger.info("=" * 60)
502             logger.info(f"LIGHTGBM - ITERA  O {iteration + 1}/{args.train_iterations}")
503             logger.info("=" * 60)
504
505             lgb_cv = kfold_cv(train_lightgbm, X_train, y_train, args.task,
                    min(args.cv_folds, 3))
506             lgb_model, lgb_params = train_lightgbm(X_train, y_train, args.task)
507             lgb_test_metrics, y_pred, y_prob = save_model_artifacts(
508                 f"lightgbm_iter{iteration+1}", lgb_model, lgb_params, lgb_cv,
509                 X_test, y_test, labels, avg, le,
510                 args.out_models, args.out_reports
511             )
512             all_lgb_metrics.append(lgb_test_metrics)
513             all_lgb_predictions.append((y_pred, y_prob))
514
515             logger.info(f"LightGBM Itera   o {iteration + 1} conclu da!")
516             logger.info(f"Acur cia: {lgb_test_metrics['accuracy']:.4f}, F1:
                    {lgb_test_metrics['f1']:.4f}")
517
518         # ============================================================
519         # CRIAR RELAT RIOS COMPARATIVOS
520         # ============================================================
521         logger.info("=" * 70)
522         logger.info("GERANDO RELAT RIOS COMPARATIVOS")
523         logger.info("=" * 70)
524
525         comparison_data = create_comparison_report(
```

```
            all_xgb_metrics, all_lgb_metrics, X_test, y_test, labels, le,
                args.out_reports
        )

        # ==========================================================
        # SELE   O DOS MELHORES MODELOS
        # ==========================================================
        logger.info("=" * 70)
        logger.info("SELECIONANDO MELHORES MODELOS")
        logger.info("=" * 70)

        # Encontrar melhores itera   es
        xgb_best_idx = np.argmax([m["accuracy"] for m in all_xgb_metrics])
        lgb_best_idx = np.argmax([m["accuracy"] for m in all_lgb_metrics])

        # Copiar melhor XGBoost
        best_xgb_source = os.path.join(args.out_models,
            f"xgboost_iter{xgb_best_idx+1}.joblib")
        best_xgb_dest = os.path.join(args.out_models, "xgboost_best.joblib")
        shutil.copy2(best_xgb_source, best_xgb_dest)
        logger.info(f"Melhor XGBoost: iter {xgb_best_idx+1} -> xgboost_best.joblib")

        # Copiar melhor LightGBM
        best_lgb_source = os.path.join(args.out_models,
            f"lightgbm_iter{lgb_best_idx+1}.joblib")
        best_lgb_dest = os.path.join(args.out_models, "lightgbm_best.joblib")
        shutil.copy2(best_lgb_source, best_lgb_dest)
        logger.info(f"Melhor LightGBM: iter {lgb_best_idx+1} -> lightgbm_best.joblib")

        # ==========================================================
        # RESUMO FINAL
        # ==========================================================
        logger.info("=" * 70)
        logger.info("TREINAMENTO CONCLUDO!")
        logger.info("=" * 70)

        best_xgb_acc = all_xgb_metrics[xgb_best_idx]["accuracy"]
        best_lgb_acc = all_lgb_metrics[lgb_best_idx]["accuracy"]

        logger.info(f"MELHOR XGBOOST: Itera   o {xgb_best_idx+1} - Acur cia:
            {best_xgb_acc:.4f}")
        logger.info(f"MELHOR LIGHTGBM: Itera   o {lgb_best_idx+1} - Acur cia:
            {best_lgb_acc:.4f}")
        logger.info(f"RELAT RIOS SALVOS EM: {args.out_reports}")
        logger.info(f"MODELOS SALVOS EM: {args.out_models}")

        logger.info("[OK] Processo completo! Verifique os relat rios em 'reports/'.")

    except Exception as e:
        logger.error(f"Erro no processo de treinamento: {e}")
        raise

if __name__ == "__main__":
    main()
```